D3Tree: Learning to Prune Block-Diffusion Draft Trees for Efficient Verification

Hong Nie[1,†], Zhangchi Gao[2], Jun Yu[1,*], Bo Wang[2], Jianqing Sun[2] and Shengping Liu[2]
[1]University of Science and Technology of China [2]Unisound AI Technology Co., Ltd.
gme.niehong@mail.ustc.edu.cn, harryjun@ustc.edu.cn
{gaozhangchi, wangbo, sunjianqing, liushengping}@unisound.com

arXiv:2611.11371v1 [cs.LG] 8 Nov 2026

***Abstract*—Block-diffusion drafters propose a whole block of tokens in one forward pass, and draft trees built from their position-wise marginals increase the expected accepted length of speculative decoding. Every tree node, however, occupies a query row in target verification, so under concurrent serving, low-yield branches compete with other requests for verification capacity. Existing tree methods choose the verified nodes by draft probability under a fixed or cost-selected per-request budget, and thus always retain a prefix of the drafter's ranking. We present D3Tree, which learns which nodes of a block-diffusion draft tree are worth verifying. A lightweight tree-size head with $2H+2$ parameters predicts the conditional survival of each candidate from the draft state, the candidate embedding, and its draft log-probability; thresholding the product of these scores along parent chains yields an ancestor-closed subtree whose size adapts to each request and decoding round. The head is trained on a frozen drafter with conditional labels on the full candidate tree, and an optional joint mode additionally steers the drafter toward the retained reference path. At serving time, D3Tree verifies whole trees under light load and, under heavy load, allocates one node budget across the batch by predicted survival, sized to the matrix-multiplication tiles of the target. All retained trees are verified in a single ancestor-masked target pass, while tree construction, pruning, and scheduling run on the GPU and overlap with the host. Across six vision-language verifiers and 17 text and multimodal benchmarks, D3Tree is the fastest of the compared verification strategies in 70 of 72 load settings, achieving 3.2–4.0$\times$ speedup over autoregressive decoding for a single request and 1.3–1.8$\times$ at 64 concurrent requests, where fixed-budget trees fall below autoregressive speed. Under a time-varying arrival trace, it also attains the lowest mean end-to-end latency in every arrival phase.**

***Index Terms*—Speculative decoding, block diffusion, draft trees, tree verification, large language model inference**

I. Introduction

Autoregressive decoding of large language models [1]–[4] is bound by memory bandwidth: every generated token requires a full pass over the model weights [5]. Speculative decoding reduces this cost by letting an inexpensive drafter propose several tokens that the target model verifies in a single forward pass [6], [7]. Its speedup is governed by two quantities: the number of draft tokens accepted per round and the cost of producing and verifying them. Parallel drafters make this trade-off explicit, since additional candidates are nearly free to propose, yet every candidate submitted for verification consumes target-model capacity.

DFlash [8] is a block-diffusion drafter that predicts a block of future tokens in one forward pass, conditioned on target-model features. Its predictions are position-wise: the distribution at a given position does not depend on the tokens realized at earlier positions of the same block. A long linear draft therefore drifts away from the target continuation, and the drift compounds, because a position contributes only if all of its predecessors are accepted. As the probability of reaching a deep position is the product of per-position match probabilities, the accepted length saturates quickly, however inexpensive the draft.

Draft trees exploit the same parallel output more fully. DDTree [9] builds the B most probable prefixes from the position-wise marginals, so that alternative branches can recover where the top-1 chain fails. The target model, however, must process every node of the tree although only one path is committed, and each request contributes its root and all B nodes to the verification batch. Under concurrency, low-yield branches therefore compete with other requests for verification capacity, and DDTree itself reports that the accepted length can keep growing after the speedup has peaked [9]. CaDDTree [10] selects the budget with a latency model, but the model describes a single request: the selected tree does not shrink when concurrent requests compete for verification, and the nodes are still ranked by the drafter's own probabilities, so the verified set is always a prefix of that ranking.

DSpark [11] relies on a different signal. It trains a confidence head to estimate conditional acceptance and lets a scheduler decide how long a linear prefix to verify. Learned survival is a natural basis for deciding how much to verify, but trees raise a question that chains do not: branches at the same depth can differ in value, so a single length cannot express which nodes deserve verification. Fig. 1 contrasts how these methods select the nodes to verify.

We propose *Block **D**iffusion **D**ynamic **D**raft **Trees*** (D3Tree), which learns which nodes of a block-diffusion draft tree to verify and adapts the verification workload to the serving load (Fig. 2). A lightweight *tree-size head* predicts, for every candidate node, the probability that it matches the target continuation given that its parent does. Multiplying these scores along parent chains and thresholding the product yields an ancestor-closed subtree. Its size equals the number of sur-

[†]This work was achieved during Hong Nie's internship at Unisound AILab under the mentorship of Zhangchi Gao (+1) and Bo Wang (+2).
[*]Corresponding author: harryjun@ustc.edu.cn

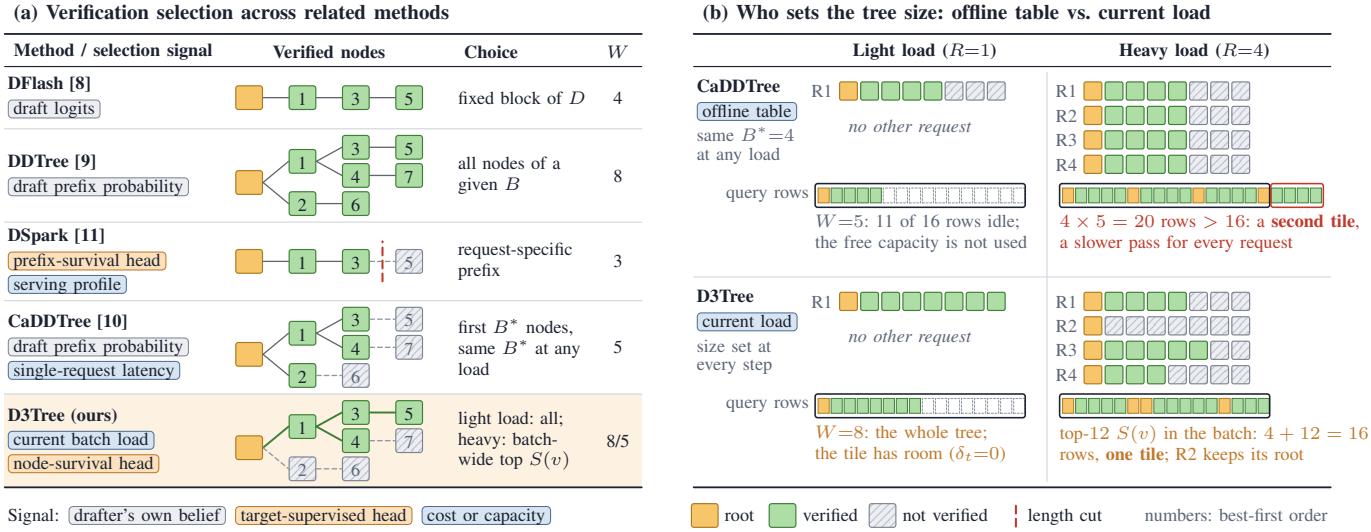

Fig. 1. Verification selection in closely related methods. (a) Each method on one candidate space; numbers give the best-first order by draft path score, and W counts target query rows, including the root. Chips mark the selection signal. (b) CaDDTree and D3Tree under light load (one request) and heavy load ($R = 4$ requests in one target pass whose matrix multiplications process tiles of $s = 16$ rows); below the trees, each step's query rows fill the tiles (dashed: idle; red: a second tile). CaDDTree sizes each tree from a latency table profiled offline for a single request, regardless of the current load, so here it verifies $B^* = 4$ nodes per request at either load: rows stay idle under light load and spill into a second tile under heavy load. D3Tree sets the size from the current load: the whole tree when the tile has room, and otherwise the 12 nodes of highest survival across the batch, which fill one tile.

viving nodes, so a single construction budget yields different verification workloads across requests and decoding rounds, and a confident deep branch can be retained while an uncertain shallow sibling is pruned.

Whether an additional verified node pays off depends on the load. With few requests, the target pass is bound by reading the model weights, and the extra query rows of a whole tree cost little; with many requests, the width of the batch determines the cost of every pass. D3Tree therefore verifies whole trees while the batch is small and, beyond a load threshold, retains the nodes of highest survival across the entire batch, up to a node budget aligned with the target's matrix-multiplication tiles. All retained trees are verified in one ancestor-masked target pass, while tree construction, pruning, and scheduling run on the GPU and overlap with the host.

We evaluate D3Tree on six vision-language verifiers from the Qwen3.5, InternVL3.5, and Gemma 4 families and on 17 text and multimodal benchmarks, against linear, dynamically truncated, fixed-budget tree, and cost-selected tree verification on the same runtime. D3Tree is the fastest strategy in 70 of 72 verifier–benchmark-group–load settings. It reaches 3.2–4.0× speedup over autoregressive decoding for a single request and 1.3–1.8× at 64 concurrent requests, where fixed-budget trees fall below autoregressive speed (0.26–0.84×), and it is at least 1.15× as fast as the best cost-selected tree at every load. In serving experiments, it completes a fixed-concurrency workload first and attains the lowest mean latency in every phase of a time-varying arrival trace, and it remains the fastest strategy in most settings on an A800 and an RTX 5090.

Our contributions are as follows.

- A conditional-survival tree-size head with $2H+2$ parameters that extends learned length control from chains to branching trees; a single threshold determines both the shape and the size of the verified subtree.
- A verification-aware training scheme that fits the head on a frozen drafter with conditional labels on the full candidate tree, together with an optional joint mode that directs token learning toward the retained reference path and its first missing position.
- A load-dependent serving design that allocates one node budget across the batch by learned survival, sized to the target's matrix-multiplication tiles, verifies all retained trees in one ancestor-masked target pass, and keeps construction, pruning, and scheduling on the GPU, overlapped with the host.
- An evaluation across six verifiers, 17 benchmarks, concurrency from 1 to 64, four GPU types, and two serving workloads, including a reimplementation of cost-selected trees on the same runtime.

Section II positions D3Tree against related drafting and verification schemes, Section III presents the method, Section IV evaluates it, and Section V concludes the paper.

II. Related Work

A. Speculative Decoding and Drafter Design

Blockwise parallel decoding [12] first predicted several future tokens and verified them in a single pass of the model. Speculative decoding and speculative sampling [6], [7], [13] made the scheme lossless by verifying the proposals of a smaller drafter against the target distribution. Subsequent work improves the drafter through knowledge distillation [14], online adaptation [15], hierarchies of drafters [16]–[18], phrase-

level drafts [19], or retrieval and reference copying [20], [21]. Draft-free variants accelerate decoding with Jacobi-style parallel iterations [22]–[24], and SpecTr [25] verifies several drafts jointly through optimal transport. Surveys and empirical studies [26], [27] show that the end-to-end speedup depends on both the acceptance rate and the drafting latency, which motivates verifying fewer but better candidates. A second line of work attaches lightweight draft modules to the target model: parallel decoding heads in Medusa [28], sequentially dependent heads in Hydra [29], a recurrent drafter [30], feature-level autoregression in EAGLE [31], [32], multi-token prediction objectives [33], [34], speculative streaming [35], and early-exit self-speculation [36]–[38]. Discrete diffusion language models [39]–[41] and block diffusion [42] generate several tokens per step, which makes them natural parallel drafters [43]. DFlash [8] predicts a whole block in one pass from target features; its position-wise distributions make candidate generation inexpensive but disregard the tokens realized earlier in the block. DSpark [11] adds a lightweight sequential module that models these within-block transitions, whereas D-PACE [44] modifies the training objective, weighting the cross-entropy at each position by a surrogate of the accepted length. D3Tree retains the parallel proposal and the D-PACE weighting, and adds supervision tied to the tree that is actually retained for verification.

B. Draft Trees and Verification Cost

Tree-based methods distribute the verification budget over alternative continuations and evaluate them in a single pass under a tree attention mask [45]. Static trees are tuned offline for a given drafter and hardware [28], [46]; Sequoia, for instance, optimizes tree size and depth by dynamic programming under a hardware-aware cost model. EAGLE-2 [47] and OPT-Tree [48] instead adapt the tree to draft confidence at run time, and SpecExec [49] constructs very large trees for offloaded inference. For block-diffusion drafters, DDTree [9] builds probability-ranked prefixes directly from the parallel marginals. Its fixed-budget objective measures coverage under the drafter's factorized distribution, which serves only as a surrogate for coverage of the target path. CaDDTree [10] trades this coverage against draft and verification latency to select a budget in every round, using a latency model profiled for a single request. Both methods rank nodes by the same drafter probabilities and therefore verify a prefix of that ranking. D3Tree instead learns a conditional node-survival signal from reference paths and prunes the bounded tree by accumulated survival, so its retained nodes need not form a prefix of the draft ranking, and the number it retains follows the load of the whole batch rather than a single-request cost. Fig. 1 contrasts these selection mechanisms.

C. Adaptive Verification under Load

Several methods adapt how much of a linear draft to verify. Dynamic speculation lookahead [50] trains a classifier that decides when to stop drafting, SpecDec++ [51] adds an acceptance-prediction head that stops once the predicted rejection risk exceeds a threshold, and PEARL [52] adapts the draft length by overlapping drafting with verification. The confidence scheduler of DSpark [11] is the closest precedent for our approach: it predicts conditional acceptance, accumulates it along a chain, and lets a scheduler, rather than a scalar length regressor, determine the verified prefix. D3Tree extends this representation from chains to branching parent chains. Its head reads the draft position state, the candidate-token embedding, and the draft log-probability, and is trained with binary reference-match labels; the resulting scores determine which branches survive and hence how many target query rows a request requires. D3Tree retains its parallel proposal distributions: it neither adopts the sequential transition module of DSpark nor assumes that DSpark's guarantees for stochastic sampling carry over to tree pruning.

The amount worth verifying also depends on the load. Serving systems batch requests at iteration granularity [53], manage the KV cache in pages [54], and split long prefills into chunks that share each step with decoding [55]. Under such batching, verification competes with other requests for compute, and the benefit of speculation diminishes as the load grows [56]. Goodput-oriented controllers therefore adapt the speculation length to the current load [57], while long-context serving shifts the trade-off again because loading the KV cache dominates the step time [58], [59]. These systems control the length of linear drafts. D3Tree controls both the size and the shape of draft trees from learned node survival, and shares one node budget, aligned with the target's matrix-multiplication tiles, across the whole batch.

III. Method

A. Overview: From Verification Length to Tree Size

D3Tree separates three quantities: the draft horizon D, the candidate construction budget B, and the retained verification size N. The drafter fixes D, the tree builder spends at most B non-root nodes, and the tree-size head decides how many of them, $N \le B$, are verified.

For a linear draft of D positions with accepted length A,

$$\mathbb{E}[A] = \sum_{d=1}^{D} \Pr(A \ge d) = \sum_{d=1}^{D} \prod_{j=1}^{d} \varphi_j, \tag{1}$$

where φ_j is the probability that position j matches given a matched prefix. Each term is the survival probability of a prefix, which is what a verification-length rule should estimate. DSpark [11] learns conditional scores $q_1^{\mathrm{lin}}, \ldots, q_D^{\mathrm{lin}}$ for these factors; with a threshold, the verified prefix has length

$$\widehat{L}_\delta = \sum_{d=1}^{D} \mathbf{1}\left\{ \prod_{j=1}^{d} q_j^{\mathrm{lin}} \ge \delta \right\}. \tag{2}$$

Equation (2) is the chain special case of our pruning rule, not DSpark's complete serving scheduler, and our tree-size head generalizes the same conditional-survival representation. Replacing chain prefixes by tree paths turns the rule into a learned subtree whose cardinality is N; the head thus controls

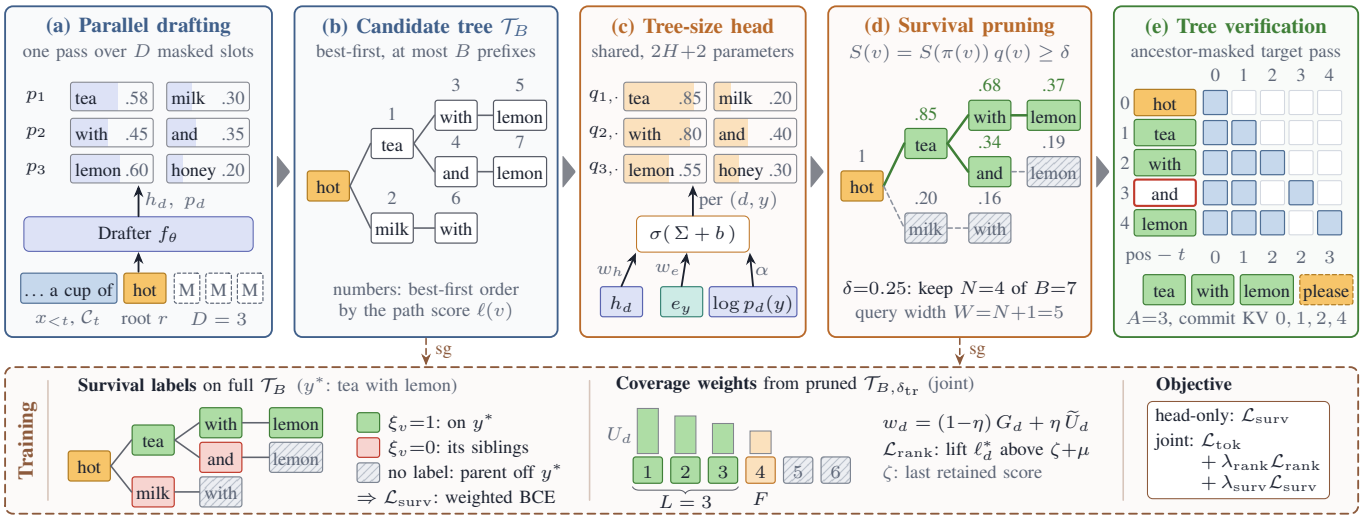

Fig. 2. D3Tree in one decoding round (top) and in training (bottom), on a running example with $B = 7$. (a) One drafter pass yields position-wise marginals p_d. (b) Best-first construction keeps the B highest-scoring prefixes; numbers give their order. (c) The tree-size head scores each depth–token pair, so repeated pairs share q. (d) Parent-chain products S (above the nodes) are thresholded: $N = 4$ nodes survive and the target verifies $W = 5$ rows. (e) Under the ancestor mask the target accepts *tea with lemon*, rejects the retained sibling *and*, emits the bonus token, and commits the KV rows of the accepted path. Survival labels come from the full tree and are all that head-only fitting uses; in the optional joint mode, the coverage L and frontier F of the pruned tree weight the token loss. Discrete construction and pruning are stop-gradient (sg). The threshold $\delta = 0.25$ is illustrative.

tree size through node survival, without regressing a depth or a node count. The rest of this section follows the pipeline of Fig. 2: Sections III-B to III-D cover drafting, scoring, and training, and Sections III-E to III-H cover the load-dependent node allocation, tree verification, efficient evaluation of the head, and the overlapped execution of a decoding round.

B. Parallel Drafting and a Bounded Candidate Tree

D3Tree combines a parallel drafter [8] with bounded best-first prefix construction [9]. Let $x_{<t}$ be the committed context and $r = x_t$ the root token produced by the target model P_ϕ, whose vocabulary is $\mathcal{V}_T$. For a horizon D, one forward pass of the drafter f_θ produces normalized states $h_d \in \mathbb{R}^H$ and position-wise distributions

$$(h_1, \ldots, h_D) = f_\theta(\mathcal{C}_t, r, [\text{MASK}]_{1:D}), \tag{3}$$

$$p_d(a) = \frac{\exp z_{d,a}}{\sum_{b \in \mathcal{V}_D} \exp z_{d,b}}, \qquad z_d = \text{LMHead}(h_d). \tag{4}$$

Here $\mathcal{C}_t$ denotes the cached target features and $\mathcal{V}_D$ the draft vocabulary. A reduced draft vocabulary is mapped to target token IDs by a fixed injection ι, and we write $p_d(\iota(a)) = p_d(a)$. Because p_d does not depend on the tokens drafted before position d, the proposal factorizes as

$$Q_\theta(y_{1:D} \mid \mathcal{C}_t, r) = \prod_{d=1}^{D} p_d(y_d \mid \mathcal{C}_t, r), \tag{5}$$

and the same p_d is shared by all branches at depth d. The tree uses these marginals to widen coverage; the learned head later decides which branches are verified.

Let $\mathcal{K}_d = \text{TopK}(p_d, K)$ with $K = \min(B, |\mathcal{V}_D|)$, and let $\text{rk}(u) \in \{1, \ldots, K\}$ be the rank of y_u in $\mathcal{K}_{d_u}$. A node v carries a token y_v, a depth d_v, a parent $\pi(v)$, and a root-excluding path $\mathcal{P}(v)$; the root is node 0. As in DDTree, a node is scored by its draft path log-probability,

$$\ell(v) = \sum_{u \in \mathcal{P}(v)} \log p_{d_u}(y_u), \qquad \ell(0) = 0, \tag{6}$$

and the bounded tree $\mathcal{T}_B$ consists of the root and the B non-root prefixes with the highest ℓ. A best-first heap finds them by lazily pushing the next sibling and the highest-ranked child of each popped node. Since

$$\ell(v) = \ell(\pi(v)) + \log p_{d_v}(y_v) \leq \ell(\pi(v)), \tag{7}$$

every parent precedes its children; ties are broken by lexicographic candidate ranks. Top-K extraction keeps the normalization over $\mathcal{V}_D$ in (4), and siblings are not renormalized. The heap runs on the GPU, one program per request, so neither the candidates nor the trees are copied to the host.

C. A Learned Tree-Size Head via Conditional Survival

For a candidate $a \in \mathcal{K}_d$ with target-token embedding $e_a \in \mathbb{R}^H$, a shared affine head with $2H+2$ parameters computes

$$g_{d,a} = w_h^\top h_d + w_e^\top e_a + \alpha \log p_d(a) + b, \tag{8}$$

$$q_{d,a} = \sigma(g_{d,a}), \qquad q(v) = q_{d_v, y_v}. \tag{9}$$

The score $q(v)$ estimates the probability that v matches the reference continuation given that its parent path matches. Scores are independent Bernoulli estimates rather than a softmax over siblings. Nodes with the same depth and token share q, but their accumulated scores differ through their parents.

The root survival and the path survival are defined recursively:

$$S(0) = 1, \qquad S(v) = S(\pi(v))q(v) = \prod_{u \in \mathcal{P}(v)} q(u). \tag{10}$$

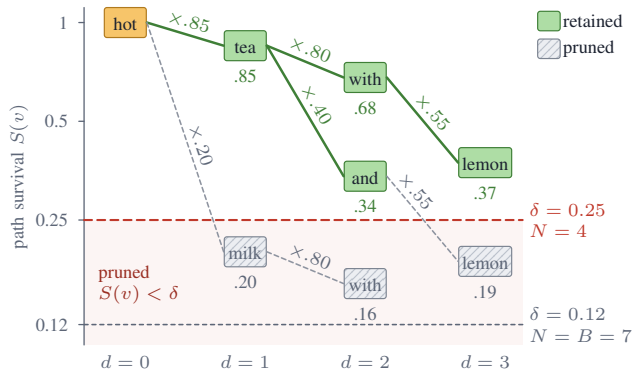

Fig. 3. Survival landscape of the example in Fig. 2. Each node sits at its depth and its path survival $S(v)$ on a log scale; edge labels give the conditional score q that multiplies S, so no node rises above its parent. At $\delta = 0.25$, $N = 4$ of the $B = 7$ candidates survive: the depth-3 path is kept, whereas *milk*, second in the best-first order, is pruned. The lower threshold 0.12 keeps a superset, here all B nodes. Both thresholds are illustrative; Section III-E chooses the threshold of each step.

For a threshold δ, the retained tree, its size, and its verification width are

$$\mathcal{T}_{B,\delta} = \{0\} \cup \{v \in \mathcal{T}_B \setminus \{0\} : S(v) \geq \delta\}, \tag{11}$$

$$N(B,\delta) = \sum_{v \in \mathcal{T}_B \setminus \{0\}} \mathbf{1}\{S(v) \geq \delta\}, \tag{12}$$

$$W(B,\delta) = N(B,\delta) + 1. \tag{13}$$

Thus B bounds construction, N counts retained draft nodes, and W counts target query rows including the root. Equation (12) is the learned size decision: even at fixed B and δ, $N = |\mathcal{T}_{B,\delta}| - 1$ follows the current node scores. For $0 < \delta \leq 1$ the test is additive in log space,

$$S(v) \geq \delta \iff \sum_{u \in \mathcal{P}(v)} -\log q(u) \leq -\log \delta, \tag{14}$$

so a branch survives while its accumulated surprisal stays within the budget $-\log \delta$. Two invariants follow directly for every non-root node v:

$$S(v) \leq S(\pi(v)), \quad \big[v \in \mathcal{T}_{B,\delta} \Rightarrow \pi(v) \in \mathcal{T}_{B,\delta}\big], \tag{15}$$

$$\delta_1 \leq \delta_2 \Rightarrow \mathcal{T}_{B,\delta_2} \subseteq \mathcal{T}_{B,\delta_1}. \tag{16}$$

The first makes every retained tree ancestor-closed; the second makes the trees for different thresholds nested. Pruning keeps the original node order and remaps parent indices. Unlike a depth cut, it can keep a deep confident branch while removing a shallow uncertain one (Fig. 3); on a single chain it reduces to (2), that is, $N = \widehat{L}_\delta$. The number of head parameters does not depend on B. Construction and pruning use different scores: ℓ only bounds which B candidates are built, and S decides which of them are verified. Section III-E chooses the threshold for every decoding step from the batch.

D. Verification-Aware Training

The head is fitted on a trained drafter whose weights stay frozen, so it can be added to an existing block-diffusion drafter without changing its proposals. All heads in our experiments are trained this way (head-only fitting). An optional joint mode (Section III-D2) additionally adapts the drafter so that useful continuations fall inside the tree that will be retained. For each training anchor, let $y^*_{1:D}$ be the reference continuation and $m_d \in \{0,1\}$ its valid-token mask; the root is not supervised. Head-only fitting takes the prepared response tokens as the reference. In joint training, the reference tokens are the target head's greedy predictions at the prepared contexts:

$$y^*_d = \iota\left(\arg\max_{a \in \mathcal{V}_D} z^{\mathrm{tgt}}_{d,a}\right), \qquad \bar{m}_d = \prod_{j=1}^{d} m_j, \tag{17}$$

where z^{tgt}_d are the target-head logits, restricted to $\mathcal{V}_D$ when the draft vocabulary is reduced. In neither case must every candidate branch be verified online.

1) Conditional node supervision: Let $\mathcal{P}^B_{y^*}$ contain the root and the valid reference prefixes that are present in the *full* candidate tree $\mathcal{T}_B$. Each candidate node receives a label mask χ_v and a target ξ_v:

$$\chi_v = \bar{m}_{d_v}\, \mathbf{1}\{\pi(v) \in \mathcal{P}^B_{y^*}\}, \qquad \xi_v = \mathbf{1}\{y_v = y^*_{d_v}\}. \tag{18}$$

A candidate that matches the reference under a matched parent is positive, its siblings are negative, and descendants of an unmatched parent are not labeled. With a positive-class weight $w_+ > 0$ and depth weights ω_d, the head minimizes the weighted binary cross-entropy

$$\mathrm{BCE}_{w_+}(q,\xi) = -w_+ \xi \log q - (1-\xi)\log(1-q), \tag{19}$$

$$\mathcal{L}_{\mathrm{surv}} = \frac{\sum_{v \neq 0} \chi_v \omega_{d_v}\, \mathrm{BCE}_{w_+}(q(v), \xi_v)}{\max\{\sum_{v \neq 0} \chi_v \omega_{d_v}, \epsilon\}}, \tag{20}$$

$$\omega_d = \exp(-(d-1)/\gamma), \qquad \gamma > 0, \tag{21}$$

where the sums run over the candidates of all anchors and $\omega_d = 1$ disables the depth decay. For this loss the log-probability input of (8) is detached, but gradients still reach h_d in joint training; target embeddings stay frozen. Because the survival loss always uses the full tree, a positive that the current head would prune is still supervised. Head-only fitting minimizes $\mathcal{L}_{\mathrm{surv}}$ alone. Since q does not depend on B, a head can be fitted on larger trees than it serves: ours are fitted on trees with $B = 31$, $w_+ = 1$, and $\gamma = 7$, and serve at $B = 15$.

2) Optional joint adaptation of the drafter: Joint training also updates the drafter, from the first epoch, with a token loss and a ranking loss that both depend on the tree the current head retains.

a) Coverage-aware token weights: During training, the tree is built and pruned with the current head at a training threshold δ_{tr}. Let L be the reference coverage of the retained tree and F the first missing position:

$$L = \max\big\{l \in \{0,\dots,D\} : \bar{m}_l = 1,\ y^*_{1:l} \in \mathcal{T}_{B,\delta_{\mathrm{tr}}}\big\}, \tag{22}$$

$$F = \min(L+1, D), \qquad U_d = (D-d+1)\bar{m}_d \mathbf{1}\{d \leq F\}, \tag{23}$$

where $\bar{m}_0 = 1$ and the empty prefix is the root. Tree construction, pruning, L, and all token weights are treated as

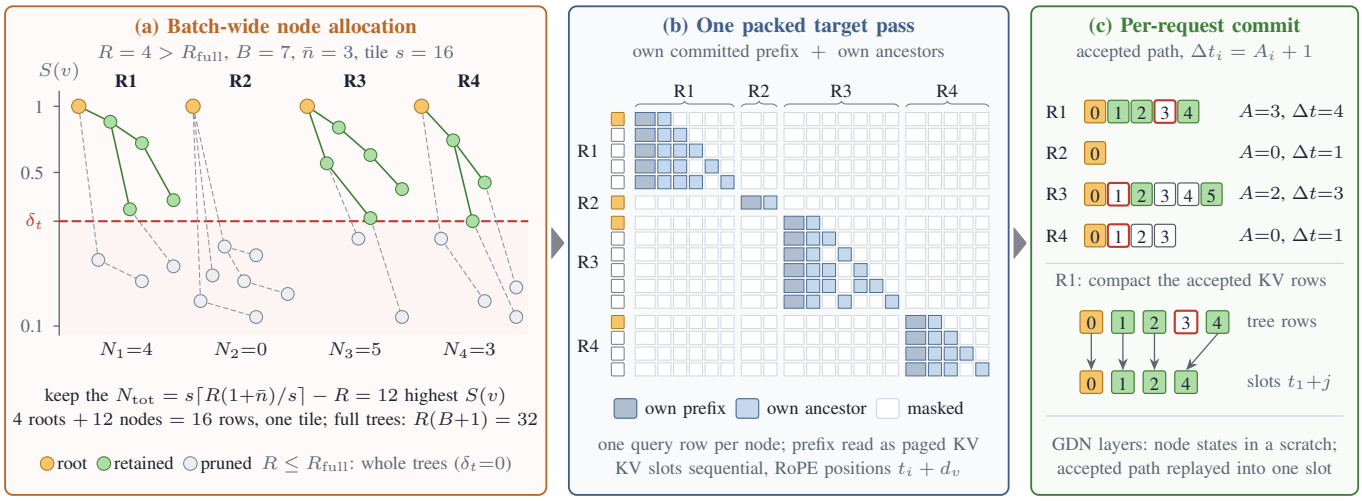

Fig. 4. Multi-request serving with D3Tree (illustrative, $R = 4$, $B = 7$, $R_{\text{full}} = 2$). (a) Every request's full candidate tree on its path survival $S(v)$ (log scale; edges join parents and children). Above R_{full} one threshold serves the whole batch: δ_t is the N_{tot}-th largest survival, with N_{tot} from (33) (here $\bar{n} = 3$ and a tile of $s = 16$ rows). Requests keep different numbers of nodes, R2 only its root, and the $R + N_{\text{tot}} = 16$ rows fill one tile instead of the $R(B+1) = 32$ of full trees. (b) All retained trees share one target pass. Each query row attends to its request's committed prefix through the block table and to its own ancestors; rows of other requests are masked. KV slots stay sequential while RoPE uses tree depth. (c) Each request accepts a path independently and commits $A_i + 1$ tokens; accepted rows are compacted, and GDN layers replay the accepted path into the request's state slot.

constants (stop-gradient). We keep the confidence-based D-PACE weighting [44],

$$\kappa_d = \begin{cases} \beta + (1-\beta)\, p_d(y_d^*), & m_d = 1, \\ 1, & m_d = 0, \end{cases} \qquad 0 < \beta \le 1, \tag{24}$$

$$G_d = m_d \sum_{j=d}^{D} m_j \prod_{k=1}^{j} \kappa_k, \tag{25}$$

and blend it with the tree utility:

$$\widetilde{U}_d = U_d \frac{\sum_j G_j}{\sum_j U_j}, \qquad w_d = (1-\eta) G_d + \eta \widetilde{U}_d, \quad \eta \in [0,1], \tag{26}$$

with $\widetilde{U} = G$ when $\sum_j U_j = 0$. The blend preserves $\sum_d w_d = \sum_d G_d$ and moves weight toward the covered path and its frontier. Over anchors i, the token loss is

$$\mathcal{L}_{\text{tok}} = \frac{\sum_{i,d} m_{i,d} w_{i,d} [-\log p_{i,d}(y_{i,d}^*)]}{\sum_{i,d} m_{i,d} + \epsilon}, \tag{27}$$

whose denominator counts valid tokens rather than their total weight.

b) Reference-prefix ranking: The token loss raises the probability of reference tokens, but a reference prefix enters the tree only if it outranks competing candidates. Let ℓ_d^* be the construction score of the reference prefix of length d and ζ the detached score of the last retained non-root node in construction order. With a margin $\mu \ge 0$,

$$\ell_d^* = \sum_{j=1}^{d} \log p_j(y_j^*), \qquad \zeta = \text{sg}[\ell(v_{\text{last}})], \tag{28}$$

$$\mathcal{L}_{\text{rank}} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \frac{\sum_d U_{i,d}\, \text{softplus}(\zeta_i + \mu - \ell_{i,d}^*)}{\max\{\sum_d U_{i,d}, \epsilon\}}, \tag{29}$$

$$\mathcal{L} = \mathcal{L}_{\text{tok}} + \lambda_{\text{rank}} \mathcal{L}_{\text{rank}} + \lambda_{\text{surv}} \mathcal{L}_{\text{surv}}, \tag{30}$$

where $\mathcal{I}$ contains the anchors with nonzero frontier utility and an empty set contributes zero. A root-only tree has $\zeta = -\infty$ and thus zero ranking loss, while its first valid frontier still receives token loss; $\lambda_{\text{rank}} = 0$ disables ranking.

The token and ranking losses use the pruned coverage, whereas the survival loss keeps the full tree. The training threshold δ_{tr} and the serving thresholds of Section III-E are chosen independently.

E. Load-Dependent Node Allocation

Verifying the full trees of R concurrent requests would occupy

$$W_{\text{full}} = R(B+1) \tag{31}$$

query rows, one root per request included, whereas the retained trees occupy $\sum_i (1 + N_i)$. Whether verifying more rows pays off depends on load. With few requests the target pass is bound by reading the model weights, so a whole tree costs little more than its root, and every pruned node only loses acceptance. With many requests the width of the batch sets the cost of the pass, and low-survival nodes delay every request in it. D3Tree therefore keeps one construction budget B and chooses the threshold of each decoding step from the number R of requests that verify drafts in that step:

$$\delta_t = \begin{cases} 0, & R \le R_{\text{full}}, \\ S_{(N_{\text{tot}})}, & R > R_{\text{full}}, \end{cases} \tag{32}$$

where $S_{(k)}$ is the k-th largest path survival among the non-root nodes of all R candidate trees and N_{tot} is a batch-wide node budget. Every request keeps $\mathcal{T}_{B,\delta_t}$ of (11) with its own $N_i(B, \delta_t)$ nodes, so each retained tree stays ancestor-closed by (15); nodes tied with $S_{(N_{\text{tot}})}$ are all kept. At light load $\delta_t = 0$

retains every node, and D3Tree verifies the same trees as a fixed-budget tree. Under load the requests compete for one budget: a request with a confident deep branch keeps more nodes than one whose candidates are uncertain, and a request whose best candidate falls below δ_t is verified with its root alone ($N_i = 0$).

The budget follows the cost profile of the target. Its matrix multiplications process the query rows in tiles of s rows, so the latency of a target pass is nearly flat within a tile and jumps when the width crosses a multiple of s (by 12–30% on an RTX 6000 Ada with $s = 64$). With a per-request allocation $\bar{n}(R)$ that decreases with load, the budget fills the last tile that the roots and $\bar{n}(R)$ nodes per request reach:

$$N_{\mathrm{tot}}(R) = s \left\lceil \frac{R\,(1 + \bar{n}(R))}{s} \right\rceil - R. \tag{33}$$

We use $B = 15$, $s = 64$, $R_{\mathrm{full}} = 8$, and $\bar{n}(R) = 7$ for $9 \le R < 24$ and $\bar{n}(R) = 5$ for $R \ge 24$. Hybrid targets with gated DeltaNet layers [60], whose verification also advances recurrent states (Section III-F), start pruning earlier: $R_{\mathrm{full}} = 4$, $\bar{n}(R) = 7$ for $5 \le R < 24$, and $\bar{n}(R) = 5$ for $R \ge 24$. These values are set on the RTX 6000 Ada; on the A800 we retune $\bar{n}(R)$ for $R \ge 16$ (Section IV-F). The budget counts draft rows only; prefill chunks scheduled in the same iteration-level batch [53], [55] are added on top of it. Without the allocation table, every step with $R > R_{\mathrm{full}}$ applies one fixed threshold to all requests ($\delta = 0.05$ by default). No node floor is used. D3Tree always verifies trees; it never falls back to linear verification.

F. Tree Verification and State Commitment

Let $\widetilde{\mathcal{T}}$ be an admitted tree and $\mathcal{A}^+(v)$ the root-inclusive ancestors of v, including v itself. As in tree-based speculative inference [45], the target assigns depth-based rotary positions [61] and applies the tree attention mask

$$\mathrm{pos}(v) = t + d_v, \quad d_0 = 0, \qquad M_{vu} = \begin{cases} 0, & u \in \mathcal{A}^+(v), \\ -\infty, & \text{otherwise.} \end{cases} \tag{34}$$

In addition, every node attends to $x_{<t}$ under the target's native attention pattern. In a batch, the trees of all requests are packed into one variable-length query. Each query row sees only the committed prefix of its own request, read from the paged KV cache [54], and its own ancestors (Fig. 4(b)); KV entries are written to consecutive slots, whereas rotary positions follow (34). Tree rows run a Triton kernel derived from vLLM's unified paged attention: M enters as an additive bias over each request's window of at most $B{+}1$ rows, and for up to 256 tree rows the prefix is additionally split into segments along the KV length. No per-step block mask is built, and the kernel reads only persistent buffers, so the whole target pass can be captured as a CUDA graph. When prefill chunks share the step, the tree rows, which the batch places first, take this kernel in one launch and the chunks a causal one. Only queries inside a bidirectional image span of a multimodal target fall back to a block-sparse FlexAttention kernel [62]. We consider greedy verification; path-dependent penalties, masks, and logits processors are not supported. The target prediction after node v is

$$\hat{y}(v) = \arg\max_{a \in \mathcal{V}_T} P_\phi\big(a \mid x_{<t}, r, y_{\mathcal{P}(v)}\big). \tag{35}$$

Starting from $v_0 = 0$, verification follows a child only if its token equals the current target prediction:

$$\begin{aligned} v_{j+1} &= \mathrm{child}(v_j, \hat{y}(v_j)) && \text{if present,} \\ b_{\mathrm{next}} &= \hat{y}(v_A) && \text{at the first miss.} \end{aligned} \tag{36}$$

The round emits the A accepted draft tokens followed by b_{next}, which becomes the next root. The KV entries of the accepted path are compacted into the committed cache:

$$\begin{gathered} \mathrm{KV}_{\mathrm{new}}[t + j] = \mathrm{KV}_{\mathrm{tree}}[v_j], \quad j = 0, \dots, A, \\ \Delta t = A + 1. \end{gathered} \tag{37}$$

Because the accepted path need not be contiguous in the packed tree, compaction follows node indices; it moves the accepted rows of all layers that share a KV cache group in one gather and one scatter. Recurrent layers of hybrid targets, such as gated DeltaNet [60] or other state-space layers [63], keep a single state slot per request. The target pass reads it without writing it; rows whose retained tree branches keep the state of every non-root node in a shared scratch, each computed from its parent's state, whereas chain-shaped rows need none. Once the accepted path is known, its tokens are replayed from the slot with the same arithmetic, order, and tiling as the target pass, so the committed state is bitwise the state of the accepted branch, and the newest convolution inputs of the path are written to the convolution window.

If tree evaluation reproduces the sequential target logits and state transitions, (36) yields the same greedy continuation as autoregressive decoding, by induction over the accepted path. This is a conditional property: batched finite-precision kernels require separate validation of numerical parity. The survival head only selects work; every emitted token is determined by the target's predictions.

G. Cached and Fused Survival Evaluation

At inference the embeddings and head weights are fixed, so (8) factors into a vocabulary table and a per-position term:

$$\nu_a = w_e^\top e_a \quad (a \in \mathcal{V}_T), \qquad o_d = w_h^\top h_d + b, \tag{38}$$

$$q_{d,a} = \sigma\big(o_d + \nu_a + \alpha \log p_d(a)\big). \tag{39}$$

The table is built once, after the drafter shares the target embeddings, and is invalidated whenever weights are reloaded or replaced. An optional fused GPU kernel then computes the position term, the table lookup, the log-probability term, and the sigmoid without materializing a $D \times K \times H$ tensor of candidate embeddings, following the IO-aware design of fused attention kernels [64], [65]. For R requests, the head arithmetic changes from

$$\underbrace{O(RDKH)}_{\text{candidate projections}} \quad \longrightarrow \quad \underbrace{O(RDH + RDK)}_{\text{per round}}, \tag{40}$$

TABLE I

Accepted length τ per benchmark at concurrency $R = 1$, greedy decoding. All strategies for a verifier share the same drafter and runtime. DFlash verifies the full linear draft; DSpark a prefix chosen by the drafter's length head; DDTree the full best-first tree of $B = 15$ nodes; CaDDTree a cost-selected prefix of its best-first tree; D3Tree prunes the $B = 15$ tree under its load-dependent node budget. Benchmarks: GSM8K, MATH-500, AIME25, HumanEval, MBPP, LiveCodeBench, MT-Bench, OmniDocBench, OCRBench-v2, DocVQA, olmOCR, CharXiv, TextVQA, MMMU-Pro, GQA, COCO Caption, and MM-MT-Bench. Highest per verifier and benchmark in bold.

Model	Strategy	Budget	Text benchmarks							Multimodal benchmarks									
			Math			Code			Chat	Document & OCR				General vision				Capt. & chat	
			GSM8K	MATH	AIME	HEval	MBPP	LCB	MT-B	ODoc	OCR2	DVQA	olmO	CXiv	TVQA	MMMU	GQA	COCO	MMMT
Q3.5-4B	DFlash	N=15	5.35	6.63	6.56	5.43	5.12	4.68	3.81	3.08	3.77	4.20	4.86	5.30	3.70	4.33	3.69	3.59	4.29
	DSpark		4.95	6.22	5.94	5.10	4.61	4.25	3.50	2.86	3.43	3.77	4.64	4.62	3.24	3.86	3.24	3.09	3.84
	DDTree		**6.09**	7.46	7.59	**6.25**	5.78	5.51	**4.58**	**3.45**	4.43	4.80	5.61	6.00	4.34	**5.17**	4.42	**4.31**	5.07
	CaDDTree	B=15	6.08	7.42	**7.70**	6.23	5.81	5.49	4.53	3.42	4.44	**4.82**	**5.64**	**6.01**	**4.35**	5.14	4.44	4.29	**5.10**
	D3Tree		**6.09**	**7.49**	7.39	**6.25**	**5.86**	**5.53**	4.52	3.44	**4.49**	4.76	5.61	5.97	4.34	5.11	**4.46**	4.27	5.04
Q3.5-9B	DFlash	N=15	5.14	6.35	6.36	5.22	4.95	4.71	3.74	2.91	3.76	4.08	4.79	4.99	3.51	4.16	3.61	3.45	4.19
	DSpark		4.64	5.68	5.61	4.80	4.53	4.17	3.34	2.69	3.37	3.71	4.34	4.50	3.02	3.70	3.09	3.01	3.72
	DDTree		5.93	**7.17**	**7.19**	6.10	5.81	**5.66**	**4.51**	3.20	**4.47**	**4.75**	**5.51**	**5.82**	**4.21**	**5.00**	4.30	**4.22**	**4.98**
	CaDDTree	B=15	5.97	7.10	7.14	6.06	**5.91**	5.54	4.48	**3.25**	4.43	4.70	5.45	5.75	4.08	4.99	4.33	4.19	4.92
	D3Tree		**6.04**	7.09	7.16	**6.11**	5.82	5.59	4.49	3.23	4.41	4.74	5.45	5.75	4.15	4.92	**4.35**	4.21	4.95
IVL3.5-4B	DFlash	N=15	5.49	7.27	4.80	3.82	4.20	3.76	3.32	2.98	3.14	3.59	4.68	4.67	2.64	5.23	3.35	3.02	3.60
	DSpark		5.47	7.00	4.15	3.46	3.89	3.36	2.97	2.96	3.07	2.98	4.21	4.08	2.28	4.95	2.74	2.52	3.13
	DDTree		6.39	**8.05**	5.51	4.57	**5.04**	**4.61**	4.01	3.48	**3.47**	4.19	**5.48**	**5.57**	**3.30**	6.27	4.04	3.77	4.24
	CaDDTree	B=15	6.34	8.04	5.50	4.56	5.01	4.57	4.03	**3.58**	3.46	**4.22**	5.29	5.39	3.21	**6.28**	4.07	3.75	**4.34**
	D3Tree		**6.48**	8.04	**5.52**	**4.59**	5.01	4.56	**4.06**	3.39	3.42	4.17	5.40	5.31	**3.30**	6.15	**4.08**	**3.78**	4.26
IVL3.5-8B	DFlash	N=15	7.08	6.14	5.43	5.38	5.04	5.29	3.16	3.21	3.46	3.69	4.26	4.75	2.68	4.75	3.31	2.93	3.73
	DSpark		6.80	6.03	5.04	5.13	4.74	5.09	2.93	3.09	3.52	3.15	3.88	4.40	2.34	4.35	2.78	2.49	3.41
	DDTree		7.64	**7.01**	6.15	6.24	5.76	6.08	3.80	3.30	**4.08**	4.23	4.88	**5.55**	3.35	5.49	**4.11**	3.67	4.47
	CaDDTree	B=15	7.66	6.82	6.02	6.28	**5.89**	**6.12**	**3.92**	3.21	3.83	**4.38**	4.92	5.53	3.35	5.46	4.05	**3.69**	**4.48**
	D3Tree		**7.68**	7.00	**6.21**	**6.35**	5.88	6.10	3.86	**3.63**	3.85	4.25	**5.02**	5.49	**3.38**	**5.52**	4.06	3.68	4.47
G4-E2B	DFlash	N=15	5.76	6.68	6.36	5.71	5.19	4.73	2.94	2.35	3.56	3.62	3.08	4.30	2.75	3.77	3.04	2.93	3.09
	DSpark		5.20	6.23	5.68	5.36	4.74	4.27	2.72	2.37	3.30	3.14	2.97	3.83	2.46	3.25	2.57	2.51	2.71
	DDTree		**6.55**	7.45	**7.34**	**6.49**	5.93	**5.59**	3.64	2.72	3.95	**4.22**	3.65	5.05	3.44	**4.55**	**3.84**	**3.71**	3.75
	CaDDTree	B=15	6.54	7.50	6.97	6.43	**5.94**	5.53	3.66	2.70	**3.99**	4.19	**3.73**	5.09	3.47	4.53	3.82	3.70	**3.77**
	D3Tree		6.53	**7.53**	7.03	6.48	5.90	5.57	**3.68**	**2.84**	3.84	4.21	3.68	**5.10**	**3.48**	4.54	3.80	**3.71**	**3.77**
G4-E4B	DFlash	N=15	6.32	7.13	6.99	6.49	5.64	5.16	3.13	2.51	3.49	3.71	3.43	4.62	2.97	4.29	3.23	2.98	3.31
	DSpark		5.77	6.71	6.50	6.26	5.22	4.84	2.87	2.52	3.21	2.84	3.17	4.06	2.37	3.76	2.60	2.50	2.85
	DDTree		**7.03**	**7.90**	**7.97**	7.02	**6.46**	5.93	**3.85**	2.86	3.96	4.34	**4.06**	**5.43**	**3.59**	4.99	**4.10**	**3.75**	**4.03**
	CaDDTree	B=15	6.97	7.86	7.93	7.05	6.40	5.93	3.83	**2.89**	4.00	4.24	**4.06**	5.42	**3.59**	5.07	4.02	3.74	3.97
	D3Tree		7.00	**7.90**	7.76	**7.08**	6.43	**5.96**	3.84	2.87	**4.02**	**4.35**	4.03	5.40	**3.59**	**5.08**	4.06	3.74	4.00

with one-time $O(|\mathcal{V}_T|H)$ projection work and $O(|\mathcal{V}_T|)$ cached scalars. This count excludes draft-logit generation, tree construction, and target verification. The fused path requires a single tensor-parallel shard and unquantized FP16/BF16 embeddings without added vocabulary; other configurations use the direct head. The factorization is exact only in exact arithmetic: reordered low-precision reductions can change scores slightly and, near the threshold, the resulting pruning decisions.

H. Overlapped Execution

A decoding round of D3Tree stays on the GPU. Candidate extraction, construction (Section III-B), survival scoring, the batch-wide threshold (32), and pruning run there, and only the retained size of each request is copied to the host, asynchronously. The host can therefore prepare step $t{+}1$ while the GPU executes step t, except for two dependencies. First, the inputs of step $t{+}1$ depend on the tokens accepted in step t. The request state kept on the GPU is authoritative: the host prepares the next step without waiting for the accepted counts and commits its own copy behind the queued work; only steps that finish or preempt requests synchronize. Second, the width of step $t{+}1$ depends on the retained sizes N_i, which exist only after drafting. The host lays out every request at its reserved width $B{+}1$, and the GPU trims the batch to the retained trees before the target pass. Verification batches of up to 1024 query rows (512 on hybrid targets) run as captured full CUDA graphs; on hybrid targets the shared node-state scratch also limits this trimming to 256 reserved rows. Steps beyond these limits, or that admit new requests or encoder inputs, wait for the retained sizes instead.

TABLE II
Speedup over AR per benchmark at concurrency $R = 1$: output throughput divided by that of AR for the same verifier, GPU, and prompts. Strategies and benchmark abbreviations as in Table I. Best per verifier and benchmark in bold.

Model	Strategy	Budget	Text benchmarks							Multimodal benchmarks								
			Math			Code			Chat	Document & OCR				General vision				Capt. & chat	
			GSM8K	MATH	AIME	HEval	MBPP	LCB	MT-B	ODoc	OCR2	DVQA	olmO	CXiv	TVQA	MMMU	GQA	COCO	MMMT
Q3.5-4B	DFlash	N=15	3.38×	4.07×	4.07×	3.40×	3.23×	2.92×	2.40×	1.99×	2.41×	2.40×	3.08×	3.26×	2.32×	2.71×	2.32×	2.25×	2.64×
	DSpark		3.25×	3.98×	3.79×	3.37×	2.97×	2.74×	2.24×	1.88×	2.24×	2.27×	3.00×	2.95×	2.07×	2.49×	2.12×	2.03×	2.48×
	DDTree		3.70×	4.34×	4.39×	3.68×	3.42×	3.23×	2.72×	2.09×	2.67×	2.55×	3.31×	3.46×	2.53×	3.04×	2.61×	2.59×	2.95×
	CaDDTree	B=15	2.77×	3.36×	3.37×	2.81×	2.57×	2.49×	2.05×	1.60×	2.03×	1.98×	2.57×	2.65×	1.92×	2.27×	1.95×	1.90×	2.19×
	D3Tree		**4.11×**	**5.04×**	**4.96×**	**4.27×**	**3.96×**	**3.70×**	**3.05×**	**2.28×**	**2.94×**	**2.76×**	**3.63×**	**3.85×**	**2.76×**	**3.33×**	**2.91×**	**2.85×**	**3.21×**
Q3.5-9B	DFlash	N=15	3.88×	4.78×	4.76×	3.94×	3.73×	3.53×	2.83×	2.15×	2.78×	2.82×	3.48×	3.67×	2.56×	3.12×	2.70×	2.59×	3.09×
	DSpark		3.51×	4.28×	4.23×	3.61×	3.41×	3.14×	2.50×	1.97×	2.48×	2.58×	3.14×	3.33×	2.20×	2.78×	2.32×	2.26×	2.75×
	DDTree		4.24×	5.10×	5.10×	4.35×	4.15×	4.00×	3.24×	2.25×	3.12×	3.07×	3.77×	4.04×	2.89×	3.55×	3.04×	2.99×	3.46×
	CaDDTree	B=15	3.53×	4.19×	4.19×	3.59×	3.49×	3.24×	2.63×	1.90×	2.57×	2.56×	3.11×	3.27×	2.31×	2.92×	2.55×	2.46×	2.85×
	D3Tree		**4.40×**	**5.16×**	**5.19×**	**4.45×**	**4.24×**	**4.04×**	**3.29×**	**2.31×**	**3.14×**	**3.11×**	**3.81×**	**4.08×**	**2.91×**	**3.57×**	**3.13×**	**3.03×**	**3.51×**
IVL3.5-4B	DFlash	N=15	4.15×	5.48×	3.72×	2.90×	3.22×	2.87×	2.56×	2.10×	2.02×	1.57×	3.37×	2.53×	1.50×	3.52×	**1.87×**	2.08×	2.54×
	DSpark		3.98×	4.97×	3.15×	2.57×	2.87×	2.48×	2.15×	1.94×	1.93×	1.39×	2.82×	2.24×	1.27×	3.17×	1.50×	1.70×	2.11×
	DDTree		4.13×	5.26×	3.81×	3.09×	3.40×	3.06×	2.73×	2.12×	1.88×	**1.62×**	3.42×	**2.61×**	1.38×	3.48×	1.75×	2.18×	2.57×
	CaDDTree	B=15	3.14×	3.87×	2.78×	2.26×	2.54×	2.30×	2.03×	1.72×	1.62×	1.39×	2.55×	2.04×	1.32×	2.93×	1.50×	1.70×	2.08×
	D3Tree		**4.67×**	**5.62×**	**4.07×**	**3.37×**	**3.65×**	**3.32×**	**2.98×**	**2.27×**	**2.09×**	1.60×	**3.70×**	2.59×	**1.66×**	**3.82×**	**1.87×**	**2.37×**	**2.80×**
IVL3.5-8B	DFlash	N=15	5.42×	4.79×	4.27×	4.17×	3.95×	4.09×	2.49×	2.39×	2.32×	1.80×	3.24×	3.25×	1.68×	3.47×	2.00×	2.12×	2.78×
	DSpark		5.11×	4.60×	3.85×	3.91×	3.63×	3.86×	2.20×	2.11×	2.21×	1.66×	2.78×	2.82×	1.42×	3.11×	1.74×	1.78×	2.46×
	DDTree		5.56×	5.21×	4.62×	4.60×	4.29×	4.47×	2.85×	2.34×	**2.54×**	1.85×	3.51×	3.54×	1.87×	3.79×	**2.24×**	2.46×	3.16×
	CaDDTree	B=15	4.39×	3.97×	3.58×	3.67×	3.46×	3.59×	2.31×	1.89×	2.00×	1.59×	2.82×	2.88×	1.55×	3.02×	1.83×	1.97×	2.52×
	D3Tree		**5.77×**	**5.37×**	**4.81×**	**4.83×**	**4.53×**	**4.63×**	**2.98×**	**2.67×**	2.51×	**2.02×**	**3.75×**	**3.59×**	**1.90×**	**3.94×**	**2.24×**	**2.51×**	**3.25×**
G4-E2B	DFlash	N=15	**5.22×**	**6.63×**	5.92×	5.58×	4.97×	4.57×	2.86×	2.21×	**2.94×**	**2.37×**	2.92×	3.84×	**2.24×**	**3.72×**	2.28×	2.64×	2.88×
	DSpark		4.70×	6.03×	5.11×	5.26×	4.63×	4.10×	2.48×	1.96×	2.49×	2.00×	2.64×	3.23×	1.85×	2.71×	1.95×	1.94×	2.34×
	DDTree		5.11×	6.46×	5.96×	6.03×	**5.41×**	4.76×	3.01×	2.12×	2.59×	2.12×	2.89×	3.39×	2.09×	3.31×	**2.30×**	2.69×	3.00×
	CaDDTree	B=15	3.67×	4.35×	4.03×	3.77×	3.57×	3.24×	2.17×	1.61×	2.25×	1.91×	2.23×	2.72×	1.79×	2.62×	1.78×	2.08×	2.15×
	D3Tree		5.19×	6.60×	**6.23×**	**6.14×**	5.39×	**5.16×**	**3.28×**	**2.38×**	2.87×	2.30×	**3.15×**	**4.21×**	2.07×	3.67×	2.16×	**3.04×**	**3.23×**
G4-E4B	DFlash	N=15	5.03×	5.84×	5.70×	5.33×	4.64×	4.21×	2.61×	2.04×	2.59×	1.93×	2.82×	3.51×	1.85×	3.37×	1.92×	2.34×	2.65×
	DSpark		4.55×	5.44×	5.23×	5.05×	4.22×	3.88×	2.28×	1.84×	2.31×	1.67×	2.50×	3.06×	1.63×	2.88×	1.69×	1.96×	2.19×
	DDTree		5.17×	6.04×	6.05×	5.34×	4.98×	4.54×	3.00×	2.17×	2.69×	1.97×	3.10×	3.77×	1.98×	3.63×	2.04×	2.69×	2.97×
	CaDDTree	B=15	3.53×	4.14×	4.24×	3.71×	3.38×	3.12×	2.01×	1.54×	1.93×	1.66×	2.13×	2.70×	1.47×	2.56×	1.58×	1.93×	2.05×
	D3Tree		**5.42×**	**6.28×**	**6.25×**	**5.61×**	**5.11×**	**4.72×**	**3.10×**	**2.24×**	**2.76×**	**2.03×**	**3.16×**	**3.91×**	**2.02×**	**3.80×**	**2.09×**	**2.80×**	**3.06×**

IV. Experiments

We evaluate D3Tree as a verification strategy for a fixed block-diffusion drafter: every strategy for a verifier uses the same drafter, prompts, generation limits, and serving runtime, and only the set of draft tokens sent to the target differs. Section IV-B compares D3Tree with linear, dynamically truncated, and fixed-budget tree verification across six verifiers and six loads, and Section IV-C compares it with cost-selected trees. Section IV-D follows the allocation within a run, at a fixed concurrency and under a time-varying arrival trace, Section IV-E varies the construction budget, and Section IV-F repeats the comparison on other GPUs.

A. Experimental Setup

Verifiers and workloads. We use six frozen vision-language verifiers: Qwen3.5-4B/9B (Q3.5), InternVL3.5-4B/8B (IVL3.5), and Gemma 4 E2B/E4B (G4). The Qwen3.5 models are hybrid targets with gated DeltaNet layers. The seven text benchmarks cover mathematics, code, and dialogue: GSM8K [66], MATH-500 [67], [68], AIME25, HumanEval [69], MBPP [70], LiveCodeBench [71], and MT-Bench [72]. The ten multimodal benchmarks cover documents, charts, visual reasoning, captioning, and dialogue: OmniDocBench [73], OCRBench-v2 [74], DocVQA [75], olmOCR [76], CharXiv [77], TextVQA [78], MMMU-Pro [79], GQA [80], COCO Caption [81], and MM-MT-Bench [82].

Drafter and heads. Each verifier has a six-layer block-diffusion drafter with the DFlash architecture [8], a compressed 32,768-token vocabulary, and a draft horizon of $D = 15$ positions. The drafters are trained on one million prompts whose responses are regenerated by the corresponding verifier, and their weights are frozen in all experiments. D3Tree serves with the configuration of Section III-E: $B = 15$, whole trees up to R_{full} requests, and the tile-aligned batch-wide node budget beyond it.

Baselines. *DFlash* [8] verifies the full 15-token linear draft. *DSpark* [11] verifies a prefix whose length is chosen per

TABLE III

Speedup over AR at concurrency R and mean accepted length τ (Avg. over benchmarks and loads), greedy decoding. Strategies as in Table I. Best speedup per verifier and column in bold.

Model	Strategy	Budget	Text benchmarks							Multimodal benchmarks						
			R=1	R=4	R=8	R=16	R=32	R=64	Avg. τ	R=1	R=4	R=8	R=16	R=32	R=64	Avg. τ
Q3.5-4B	DFlash	N=15	3.35×	3.33×	2.92×	1.92×	1.31×	1.02×	5.36	2.54×	2.39×	2.09×	1.40×	1.01×	0.78×	4.06
	DSpark	N=15	3.19×	3.31×	3.05×	2.42×	1.78×	1.40×	4.95	2.35×	2.34×	2.18×	1.82×	1.41×	1.17×	3.66
	DDTree	B=15	3.64×	3.49×	2.89×	1.83×	1.20×	0.94×	6.16	2.78×	2.55×	2.13×	1.40×	0.97×	0.75×	4.76
	CaDDTree	B=15	2.77×	2.72×	2.44×	1.86×	1.30×	1.03×	6.16	2.11×	2.05×	1.82×	1.39×	1.03×	0.83×	4.75
	D3Tree	B=15	**4.15**×	**3.78**×	**3.09**×	**2.60**×	**1.94**×	**1.56**×	5.57	**3.06**×	**2.69**×	**2.34**×	**1.89**×	**1.48**×	**1.21**×	4.37
Q3.5-9B	DFlash	N=15	3.92×	3.39×	2.61×	1.88×	1.36×	1.00×	5.24	2.89×	2.24×	1.82×	1.38×	1.04×	0.77×	3.93
	DSpark	N=15	3.53×	3.27×	2.90×	2.33×	1.73×	1.42×	4.70	2.58×	2.36×	2.09×	1.73×	**1.41**×	**1.14**×	3.52
	DDTree	B=15	4.31×	3.50×	2.80×	1.82×	1.27×	0.93×	6.05	3.22×	2.51×	2.01×	1.35×	0.96×	0.72×	4.63
	CaDDTree	B=15	3.55×	3.06×	2.71×	1.87×	1.39×	1.04×	6.06	2.65×	2.23×	1.91×	1.37×	1.04×	0.79×	4.61
	D3Tree	B=15	**4.40**×	**3.65**×	**3.04**×	**2.53**×	**1.84**×	**1.46**×	5.45	**3.26**×	**2.62**×	**2.24**×	**1.88**×	1.40×	1.13×	4.24
IVL3.5-4B	DFlash	N=15	3.56×	3.23×	2.66×	1.67×	1.24×	0.95×	4.69	2.37×	2.04×	1.68×	1.24×	1.09×	0.93×	3.70
	DSpark	N=15	3.17×	3.09×	2.83×	2.19×	1.61×	1.31×	4.32	2.05×	1.90×	1.70×	1.52×	1.29×	1.21×	3.29
	DDTree	B=15	3.64×	3.02×	2.16×	1.19×	0.77×	0.63×	5.44	2.36×	1.89×	1.45×	1.01×	0.77×	0.64×	4.36
	CaDDTree	B=15	2.70×	2.56×	2.36×	1.73×	1.36×	1.03×	5.44	1.92×	1.75×	1.51×	1.26×	1.13×	0.99×	4.36
	D3Tree	B=15	**3.95**×	**3.63**×	**3.10**×	**2.45**×	**1.83**×	**1.45**×	5.07	**2.54**×	**2.24**×	**1.86**×	**1.63**×	**1.41**×	**1.28**×	4.16
IVL3.5-8B	DFlash	N=15	4.17×	3.57×	3.05×	2.09×	1.60×	1.17×	5.39	2.54×	2.12×	1.86×	1.36×	1.19×	0.97×	3.69
	DSpark	N=15	3.88×	3.47×	3.09×	2.51×	1.87×	1.45×	5.12	2.24×	2.06×	1.87×	1.63×	1.41×	1.23×	3.35
	DDTree	B=15	4.51×	3.67×	2.77×	1.58×	1.04×	0.79×	6.11	2.76×	2.10×	1.67×	1.14×	0.87×	0.74×	4.33
	CaDDTree	B=15	3.57×	3.01×	2.70×	1.93×	1.56×	1.16×	6.09	2.23×	1.93×	1.65×	1.30×	1.15×	1.03×	4.33
	D3Tree	B=15	**4.70**×	**3.97**×	**3.42**×	**2.74**×	**1.99**×	**1.59**×	5.70	**2.87**×	**2.38**×	**2.02**×	**1.74**×	**1.44**×	**1.32**×	4.16
G4-E2B	DFlash	N=15	5.10×	4.88×	3.67×	2.56×	1.71×	1.32×	5.33	2.84×	2.48×	1.95×	1.46×	1.11×	0.92×	3.23
	DSpark	N=15	4.61×	4.52×	3.71×	2.86×	2.09×	1.63×	4.93	2.33×	2.18×	1.94×	1.64×	1.34×	1.21×	2.88
	DDTree	B=15	5.24×	3.51×	1.82×	0.79×	0.51×	0.38×	6.13	2.68×	1.79×	1.03×	0.49×	0.28×	0.25×	3.91
	CaDDTree	B=15	3.54×	3.41×	2.79×	2.31×	1.59×	1.35×	6.08	2.13×	1.95×	1.74×	1.42×	1.15×	1.07×	3.90
	D3Tree	B=15	**5.42**×	**5.12**×	**4.12**×	**3.32**×	**2.61**×	**2.07**×	5.69	**2.95**×	**2.76**×	**2.33**×	**1.98**×	**1.67**×	**1.52**×	3.70
G4-E4B	DFlash	N=15	4.77×	4.25×	3.19×	2.29×	1.47×	0.93×	5.83	2.53×	2.19×	1.75×	1.33×	1.02×	0.84×	3.43
	DSpark	N=15	4.38×	3.94×	3.09×	2.45×	1.75×	1.15×	5.46	2.20×	1.95×	1.74×	1.51×	1.28×	1.12×	3.00
	DDTree	B=15	5.02×	3.00×	1.56×	0.70×	0.43×	0.34×	6.56	2.74×	1.56×	0.84×	0.42×	0.21×	0.17×	4.12
	CaDDTree	B=15	3.45×	3.32×	2.70×	2.12×	1.50×	1.22×	6.55	1.97×	1.82×	1.56×	1.28×	1.02×	0.85×	4.12
	D3Tree	B=15	**5.21**×	**4.58**×	**3.60**×	**2.86**×	**2.08**×	**1.63**×	6.10	**2.83**×	**2.39**×	**1.93**×	**1.66**×	**1.39**×	**1.14**×	3.90

request by the drafter's learned length head with the chain rule (2) and threshold 0.3; it reproduces DSpark's learned length selection, not its serving scheduler. *DDTree* [9] verifies the full best-first tree of $B \in \{15, 31, 63\}$ nodes. It uses the same tree builder as D3Tree, so its trees at $B = 15$ are exactly those that D3Tree prunes. Tables I–III report $B = 15$, the construction budget of D3Tree. *CaDDTree* [10] is our reimplementation of cost-selected trees on the same runtime. For each request and round it grows the best-first tree under the score (6) while the estimated throughput $\big(1 + \sum_v e^{\ell(v)}\big)/T(n; c)$ increases, where the sum runs over the first n nodes in best-first order, $\ell(v)$ is the path score (6), and $T(n; c)$ is the single-request round latency for n nodes at context length c. We profile T on every verifier and GPU at $n \in \{0, 15, \ldots, 127\}$ and c from 1K to 16K tokens and fit it monotone and convex, following the original method. We run budgets $B \in \{15, 31, 63, 127\}$.

Protocol and metrics. Each benchmark supplies at most 100 prompts, outputs are capped at 1024 tokens with the native end-of-sequence token, and decoding is greedy. Fixed-load sweeps use client concurrency $R \in \{1, 4, 8, 16, 32, 64\}$, with three warm-up requests before every measurement. We report the accepted length $\tau = \mathbb{E}[A] + 1$, the mean number of tokens committed per verified draft including the bonus token, and output throughput over the elapsed benchmark time, including prefill. The speedup of a benchmark group divides the unweighted mean throughput over its benchmarks by the same mean of autoregressive decoding (AR) for the same verifier, GPU, load, and prompts. Avg. τ averages over the benchmarks of a group and the six loads.

Hardware and runtime. All methods, AR included, run on one build of vLLM [54] with CUDA graphs and at most 64 concurrent sequences; speculative methods use the FlexAttention [62] backend, with tree rows on the Triton tree-attention kernel of Section III-F. Q3.5-4B, IVL3.5-4B, and G4-E2B run on RTX 6000 Ada GPUs (48 GB), and Q3.5-9B, IVL3.5-8B, and G4-E4B on RTX A6000 GPUs (48 GB). All strategies of one verifier run on the same machine, so every speedup compares measurements on the same GPU type. Section IV-F repeats the comparison for Q3.5-4B, IVL3.5-4B,

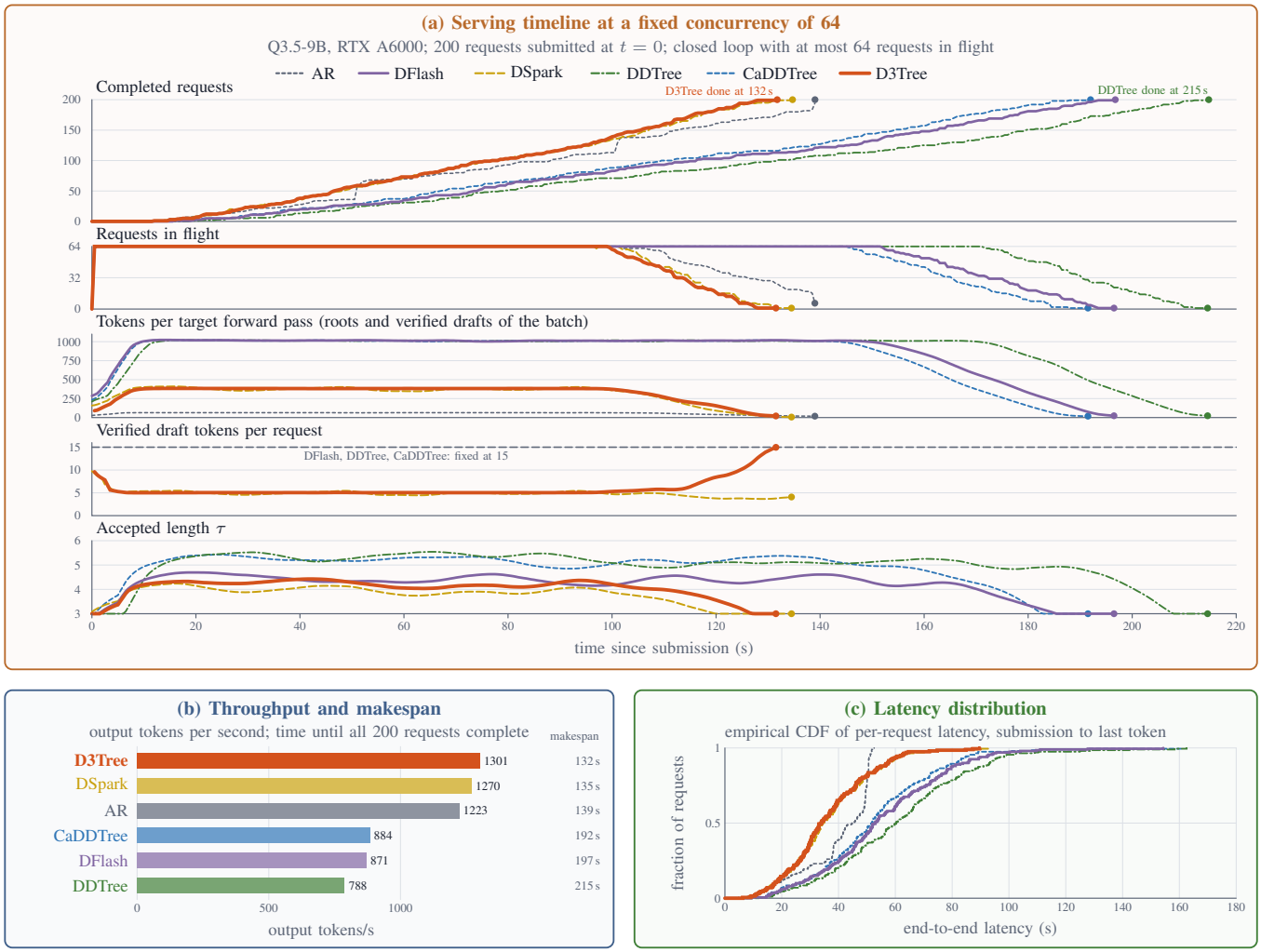

Fig. 5. Every strategy serving 200 requests at a fixed concurrency of 64 (Q3.5-9B, RTX A6000; all requests are due at once, as in the sweeps of Table III). (a) Each curve ends, with a dot, when the last request of that run finishes. From top: requests completed; requests in flight; tokens per target pass, i.e., the roots and verified draft tokens of all decoding requests; verified draft tokens per request and decoding round; and accepted length τ (sliding means over about 12, 12, and 28 s). (b) Output throughput over the run and the time until all 200 requests have finished. (c) Distribution of the latency from sending a request to its last token.

and G4-E2B on an RTX 5090 (32 GB) and for all six verifiers on an A800 (80 GB).

B. Main Results

Single request. With few concurrent requests, the target pass is bound by reading the weights, and D3Tree verifies whole trees. Tables I and II report the accepted length and speedup of every strategy per benchmark at $R = 1$. Table IV confirms that at $R = 1$ it verifies all 15 nodes per tree and matches the accepted length of DDTree $B = 15$ (for Q3.5-4B, 6.16 vs. 6.18 tokens on text and 4.75 vs. 4.76 on multimodal benchmarks). Its single-request speedup over all 17 benchmarks is 3.17–4.03×, up to 12% above DDTree $B = 15$ on the same trees; this margin comes from the GPU-resident construction and the overlapped execution of Section III-H, not from pruning. Across the 102 verifier–benchmark entries, the accepted length of D3Tree differs from that of DDTree $B = 15$ by 0.05 tokens on average, and exceeds that of DFlash's linear draft by 0.28–0.99 tokens. D3Tree is the fastest strategy on 91 entries, with speedups of 1.60–6.60× over AR, and is faster than DFlash on 95. Eight of the eleven exceptions are G4-E2B, the smallest verifier, on which DFlash is ahead on six benchmarks by up to 0.17×; on the other three, DDTree $B = 15$ is ahead by at most 0.03×. Larger DDTree budgets raise the accepted length further ($B = 63$ reaches 7.0 tokens on text for Q3.5-4B), but they are faster than D3Tree in at most four settings at $R = 1$, by 0.01–0.11×, and fall further behind under load.

Concurrent requests. Table III reports the speedup of every strategy at all six loads. D3Tree attains the highest speedup in 70 of the 72 verifier–group–load settings; the two exceptions are DSpark on the multimodal benchmarks of Q3.5-9B at $R = 32$ and $R = 64$, each ahead by 0.01×. As the load grows, the cost of a target pass follows the number of verified

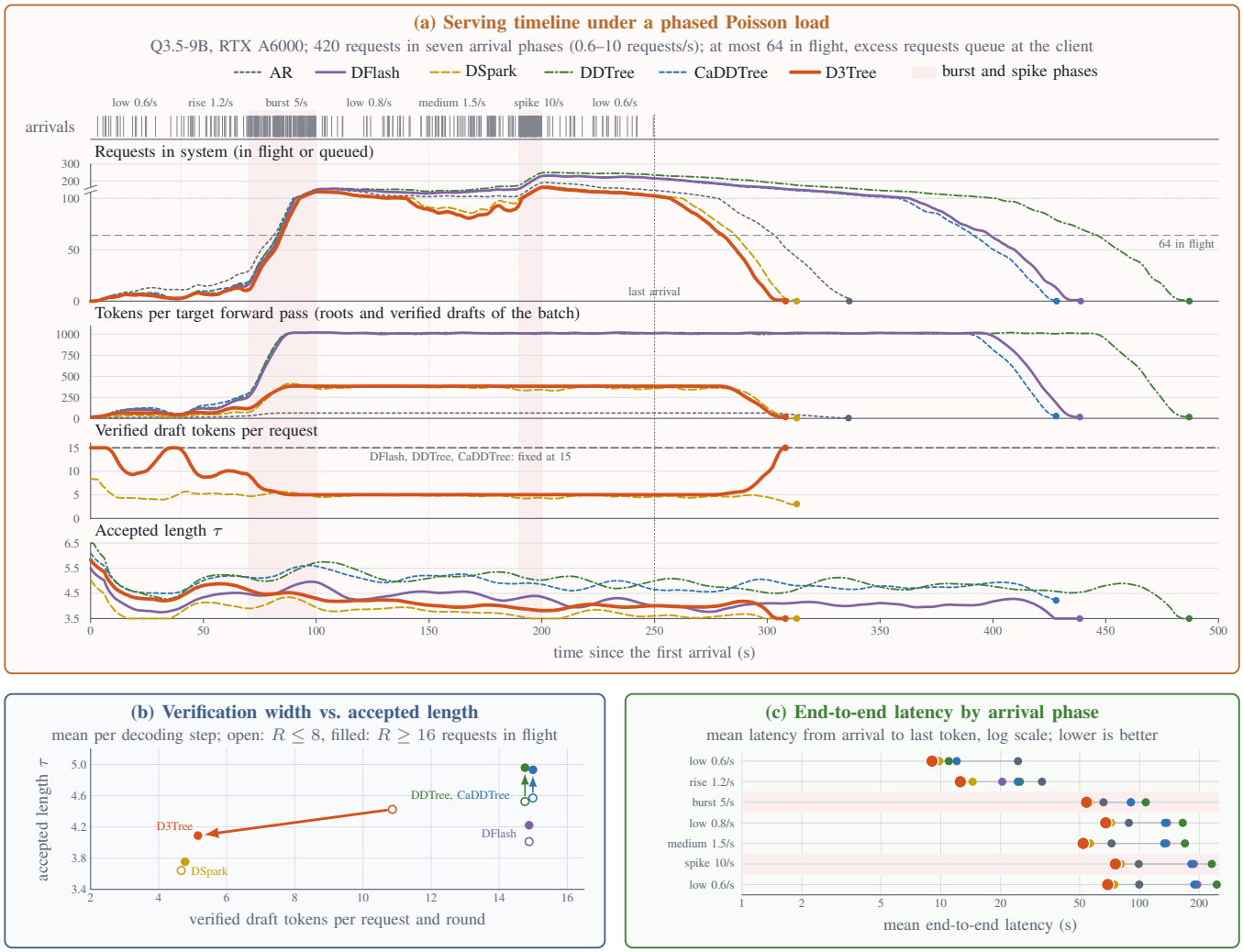

Fig. 6. Every strategy serving the same time-varying load. (a) Each curve ends, with a dot, when the last request of that run finishes. From top: one tick per arriving request; requests in the system (in flight or waiting at the client; 4-s moving average; the axis is compressed above 100); and, as in Fig. 5, tokens per target pass, verified draft tokens per request and decoding round, and accepted length τ. (b) Mean verified draft tokens and accepted length τ over the decoding steps with at most 8 (open) and at least 16 (filled) requests in flight; AR is omitted. (c) Mean end-to-end latency, from arrival to the last token, of the requests arriving in each phase (log scale).

rows, and fixed trees lose most of their advantage: DDTree falls below AR at $R = 64$ on every verifier (0.26–0.84× over all benchmarks). DSpark, which shortens its chains, is the strongest baseline here. D3Tree prunes once the load exceeds R_{full}: at $R = 8$ for the hybrid Qwen3.5 targets and at $R = 16$ for the others, it keeps 7.4–8.5 nodes per tree, and 5.8–7.0 at $R \geq 32$ (Table IV). Pruning more than halves the verified rows but costs only 0.6–0.8 accepted tokens, because the batch-wide budget keeps the highest-scoring nodes. At $R = 64$, D3Tree reaches 1.28–1.81× over all benchmarks, compared with 1.14–1.44× for the best baseline of each verifier, and 1.36–1.59× the throughput of DFlash's linear verification of the same drafts.

C. Comparison with Cost-Selected Trees

Table IV compares the strategies at every budget we ran. No fixed budget is best across loads: the fastest fixed configuration is DDTree ($B = 15$ or 31) at $R = 1$, DDTree $B = 15$ or the full DFlash draft ($N = 15$) at $R = 4$, and DFlash truncated to $N = 7$ or $N = 5$ tokens from $R = 16$ on. D3Tree with $B = 15$ is faster than the best fixed budget of each verifier at every load in 34 of the 36 verifier–load settings; the exceptions are G4-E2B and G4-E4B at $R = 1$, where DDTree $B = 31$ is ahead by at most 0.04×.

CaDDTree selects the size of each tree from a single-request latency model, so the size it selects does not depend on load. Its accepted length is constant across loads, as is its tree size: with $B = 127$ it keeps about 31 nodes per tree for Q3.5-4B and 74–81 for IVL3.5 and G4 at every load. Larger budgets therefore raise τ by up to 0.8 tokens but lose throughput as R grows, and $B = 15$ is its fastest budget at every load of $R \geq 8$. At $B = 15$ the profiled latency rises so little over 15 nodes that the cost-selected cutoff never binds: CaDDTree verifies the same full trees as DDTree $B = 15$, with the same

TABLE IV

Every strategy at every verification budget over all 17 benchmarks: speedup over AR, and accepted length τ / verified draft tokens per request and decoding round (tree nodes, root excluded), at each concurrency R. DFlash verifies the first N tokens of the linear draft (budget N); DSpark a prefix of at most $N = 15$ tokens chosen by the drafter's length head; DDTree verifies the full best-first tree of B nodes, CaDDTree a cost-selected prefix of its best-first tree, and D3Tree prunes the B-node tree under its load-dependent node budget. Best speedup per verifier and column in bold.

Model	Strategy	Budget	Speedup over AR at concurrency R						Accepted length τ / verified draft tokens at concurrency R					
			1	4	8	16	32	64	1	4	8	16	32	64
Q3.5-4B	DFlash	N=5	2.44×	2.41×	2.34×	2.00×	1.58×	1.29×	3.67 / 5.0	3.67 / 5.0	3.67 / 5.0	3.66 / 5.0	3.66 / 5.0	3.67 / 5.0
		N=7	2.66×	2.65×	2.52×	2.04×	1.48×	1.19×	4.06 / 7.0	4.05 / 7.0	4.07 / 7.0	4.05 / 7.0	4.06 / 7.0	4.07 / 7.0
		N=15	2.88×	2.78×	2.45×	1.63×	1.15×	0.89×	4.61 / 15.0	4.58 / 15.0	4.60 / 15.0	4.59 / 15.0	4.60 / 15.0	4.59 / 15.0
	DSpark	N=15	2.71×	2.74×	2.55×	2.09×	1.58×	1.28×	4.19 / 5.5	4.20 / 5.5	4.20 / 5.5	4.19 / 5.5	4.19 / 5.5	4.18 / 5.5
	DDTree	B=15	3.14×	2.94×	2.46×	1.59×	1.07×	0.84×	5.35 / 15.0	5.35 / 15.0	5.34 / 15.0	5.33 / 15.0	5.32 / 15.0	5.35 / 15.0
		B=31	3.20×	2.70×	1.77×	1.08×	0.73×	0.56×	5.76 / 31.0	5.75 / 31.0	5.77 / 31.0	5.75 / 31.0	5.79 / 31.0	5.74 / 31.0
		B=63	2.89×	1.73×	1.03×	0.61×	0.41×	0.31×	6.05 / 63.0	6.05 / 63.0	6.07 / 63.0	6.07 / 63.0	6.09 / 63.0	6.06 / 63.0
	CaDDTree	B=15	2.39×	2.33×	2.08×	1.60×	1.16×	0.92×	5.35 / 15.0	5.34 / 15.0	5.32 / 15.0	5.34 / 15.0	5.32 / 15.0	5.32 / 15.0
		B=31	2.37×	2.16×	1.70×	1.16×	0.82×	0.64×	5.65 / 26.0	5.67 / 26.0	5.63 / 26.0	5.65 / 26.0	5.67 / 26.0	5.64 / 26.0
		B=63	2.34×	2.10×	1.55×	1.03×	0.73×	0.58×	5.72 / 30.5	5.72 / 30.5	5.68 / 30.5	5.72 / 30.5	5.70 / 30.5	5.71 / 30.5
		B=127	2.21×	1.91×	1.44×	0.96×	0.70×	0.55×	5.72 / 30.9	5.73 / 30.9	5.70 / 30.9	5.71 / 30.9	5.73 / 30.9	5.72 / 30.9
	D3Tree	B=15	**3.52**×	**3.14**×	**2.66**×	**2.20**×	**1.70**×	**1.37**×	5.33 / 15.0	5.35 / 15.0	4.72 / 7.5	4.75 / 7.5	4.52 / 5.9	4.52 / 5.9
Q3.5-9B	DFlash	N=5	2.74×	2.45×	2.14×	1.92×	1.47×	1.23×	3.60 / 5.0	3.60 / 5.0	3.60 / 5.0	3.61 / 5.0	3.61 / 5.0	3.60 / 5.0
		N=7	2.99×	2.76×	2.25×	2.04×	1.46×	1.18×	3.96 / 7.0	3.98 / 7.0	3.96 / 7.0	3.97 / 7.0	3.97 / 7.0	3.96 / 7.0
		N=15	3.32×	2.72×	2.16×	1.60×	1.18×	0.88×	4.47 / 15.0	4.46 / 15.0	4.48 / 15.0	4.46 / 15.0	4.47 / 15.0	4.47 / 15.0
	DSpark	N=15	2.97×	2.74×	2.44×	1.99×	1.55×	1.27×	4.00 / 5.0	4.00 / 5.0	4.02 / 5.0	4.02 / 5.0	4.01 / 5.0	3.99 / 5.0
	DDTree	B=15	3.67×	2.93×	2.35×	1.55×	1.10×	0.81×	5.22 / 15.0	5.20 / 15.0	5.21 / 15.0	5.21 / 15.0	5.21 / 15.0	5.23 / 15.0
		B=31	3.47×	2.60×	1.62×	1.07×	0.70×	0.52×	5.65 / 31.0	5.66 / 31.0	5.66 / 31.0	5.66 / 31.0	5.66 / 31.0	5.67 / 31.0
		B=63	3.16×	1.67×	1.03×	0.61×	0.40×	0.29×	5.97 / 63.0	5.98 / 63.0	5.96 / 63.0	5.98 / 63.0	5.98 / 63.0	5.98 / 63.0
	CaDDTree	B=15	3.03×	2.58×	2.25×	1.59×	1.20×	0.90×	5.19 / 15.0	5.21 / 15.0	5.19 / 15.0	5.21 / 15.0	5.21 / 15.0	5.22 / 15.0
		B=31	2.94×	2.44×	1.70×	1.20×	0.85×	0.65×	5.51 / 25.2	5.53 / 25.2	5.53 / 25.2	5.53 / 25.2	5.53 / 25.2	5.53 / 25.2
		B=63	2.81×	2.20×	1.55×	1.08×	0.74×	0.55×	5.58 / 28.7	5.57 / 28.7	5.57 / 28.7	5.60 / 28.6	5.58 / 28.7	5.58 / 28.7
		B=127	2.64×	2.16×	1.52×	1.07×	0.71×	0.51×	5.61 / 28.9	5.59 / 28.9	5.58 / 28.9	5.59 / 28.9	5.60 / 28.9	5.58 / 28.9
	D3Tree	B=15	**3.73**×	**3.05**×	**2.58**×	**2.17**×	**1.60**×	**1.28**×	5.20 / 15.0	5.20 / 15.0	4.61 / 7.4	4.62 / 7.4	4.41 / 5.8	4.40 / 5.9
IVL3.5-4B	DFlash	N=5	2.43×	2.32×	2.19×	1.89×	1.54×	1.30×	3.37 / 5.0	3.36 / 5.0	3.36 / 5.0	3.37 / 5.0	3.39 / 5.0	3.37 / 5.0
		N=7	2.63×	2.52×	2.32×	1.91×	1.43×	1.25×	3.67 / 7.0	3.68 / 7.0	3.68 / 7.0	3.68 / 7.0	3.68 / 7.0	3.69 / 7.0
		N=15	2.90×	2.59×	2.16×	1.47×	1.18×	0.95×	4.09 / 15.0	4.09 / 15.0	4.13 / 15.0	4.10 / 15.0	4.10 / 15.0	4.11 / 15.0
	DSpark	N=15	2.55×	2.45×	2.25×	1.88×	1.48×	1.27×	3.72 / 5.2	3.73 / 5.2	3.72 / 5.2	3.75 / 5.2	3.69 / 5.2	3.71 / 5.1
	DDTree	B=15	2.93×	2.41×	1.79×	1.11×	0.77×	0.64×	4.82 / 15.0	4.79 / 15.0	4.77 / 15.0	4.82 / 15.0	4.81 / 15.0	4.81 / 15.0
		B=31	3.13×	2.36×	1.46×	0.93×	0.62×	0.50×	5.18 / 31.0	5.18 / 31.0	5.18 / 31.0	5.16 / 31.0	5.16 / 31.0	5.16 / 31.0
		B=63	3.11×	1.73×	1.15×	0.70×	0.46×	0.36×	5.46 / 63.0	5.45 / 63.0	5.45 / 63.0	5.47 / 63.0	5.48 / 63.0	5.48 / 63.0
	CaDDTree	B=15	2.27×	2.12×	1.92×	1.51×	1.26×	1.02×	4.80 / 15.0	4.80 / 15.0	4.80 / 15.0	4.80 / 15.0	4.81 / 15.0	4.80 / 15.0
		B=31	2.37×	2.08×	1.58×	1.18×	0.91×	0.74×	5.16 / 30.3	5.19 / 30.3	5.15 / 30.3	5.14 / 30.3	5.13 / 30.3	5.17 / 30.3
		B=63	2.33×	1.69×	1.24×	0.85×	0.62×	0.51×	5.41 / 57.3	5.41 / 57.2	5.42 / 57.2	5.43 / 57.1	5.44 / 57.2	5.43 / 57.2
		B=127	2.16×	1.43×	0.99×	0.66×	0.49×	0.39×	5.49 / 78.4	5.53 / 78.3	5.50 / 78.3	5.53 / 78.3	5.54 / 78.2	5.53 / 78.1
	D3Tree	B=15	**3.17**×	**2.88**×	**2.46**×	**2.07**×	**1.66**×	**1.38**×	4.79 / 15.0	4.81 / 15.0	4.80 / 15.0	4.39 / 8.5	4.19 / 6.8	4.21 / 6.9
IVL3.5-8B	DFlash	N=5	2.65×	2.45×	2.25×	2.00×	1.60×	1.37×	3.50 / 5.0	3.49 / 5.0	3.51 / 5.0	3.51 / 5.0	3.51 / 5.0	3.50 / 5.0
		N=7	2.89×	2.68×	2.39×	2.06×	1.60×	1.35×	3.85 / 7.0	3.86 / 7.0	3.84 / 7.0	3.85 / 7.0	3.86 / 7.0	3.86 / 7.0
		N=15	3.24×	2.78×	2.42×	1.73×	1.42×	1.09×	4.37 / 15.0	4.38 / 15.0	4.38 / 15.0	4.38 / 15.0	4.41 / 15.0	4.40 / 15.0
	DSpark	N=15	2.94×	2.70×	2.45×	2.07×	1.66×	1.36×	4.07 / 5.6	4.09 / 5.6	4.08 / 5.6	4.08 / 5.6	4.09 / 5.6	4.07 / 5.6
	DDTree	B=15	3.52×	2.81×	2.20×	1.36×	0.96×	0.77×	5.05 / 15.0	5.04 / 15.0	5.11 / 15.0	5.07 / 15.0	5.06 / 15.0	5.07 / 15.0
		B=31	3.65×	2.69×	1.72×	1.14×	0.75×	0.61×	5.46 / 31.0	5.46 / 31.0	5.47 / 31.0	5.48 / 31.0	5.47 / 31.0	5.48 / 31.0
		B=63	3.49×	1.93×	1.36×	0.84×	0.56×	0.44×	5.77 / 63.0	5.75 / 63.0	5.74 / 63.0	5.79 / 63.0	5.76 / 63.0	5.75 / 63.0
	CaDDTree	B=15	2.81×	2.42×	2.15×	1.62×	1.37×	1.10×	5.04 / 15.0	5.05 / 15.0	5.05 / 15.0	5.07 / 15.0	5.07 / 15.0	5.04 / 15.0
		B=31	2.94×	2.44×	1.80×	1.40×	1.04×	0.84×	5.44 / 30.2	5.46 / 30.2	5.48 / 30.2	5.43 / 30.2	5.46 / 30.2	5.45 / 30.2
		B=63	2.85×	1.83×	1.41×	0.97×	0.71×	0.58×	5.71 / 57.0	5.72 / 57.0	5.74 / 57.0	5.72 / 57.0	5.74 / 57.0	5.72 / 56.9
		B=127	2.62×	1.59×	1.13×	0.79×	0.58×	0.45×	5.79 / 79.1	5.83 / 79.0	5.83 / 78.9	5.82 / 78.9	5.83 / 78.8	5.79 / 79.1
	D3Tree	B=15	**3.66**×	**3.10**×	**2.69**×	**2.25**×	**1.74**×	**1.48**×	5.08 / 15.0	5.06 / 15.0	5.06 / 15.0	4.63 / 8.2	4.45 / 6.8	4.47 / 7.0
G4-E2B	DFlash	N=5	3.14×	2.99×	2.65×	2.28×	1.86×	1.62×	3.34 / 5.0	3.35 / 5.0	3.33 / 5.0	3.34 / 5.0	3.34 / 5.0	3.34 / 5.0
		N=7	3.41×	3.37×	2.96×	2.37×	1.84×	1.55×	3.66 / 7.0	3.66 / 7.0	3.67 / 7.0	3.68 / 7.0	3.66 / 7.0	3.66 / 7.0
		N=15	3.82×	3.51×	2.75×	1.99×	1.42×	1.14×	4.11 / 15.0	4.10 / 15.0	4.07 / 15.0	4.10 / 15.0	4.08 / 15.0	4.11 / 15.0
	DSpark	N=15	3.32×	3.19×	2.76×	2.23×	1.73×	1.44×	3.72 / 4.9	3.72 / 4.9	3.73 / 5.0	3.71 / 4.9	3.73 / 4.9	3.75 / 4.9
	DDTree	B=15	3.79×	2.53×	1.40×	0.63×	0.40×	0.32×	4.82 / 15.0	4.81 / 15.0	4.83 / 15.0	4.82 / 15.0	4.84 / 15.0	4.83 / 15.0
		B=31	**4.07**×	2.38×	1.23×	0.59×	0.34×	0.28×	5.21 / 31.0	5.20 / 31.0	5.20 / 31.0	5.19 / 31.0	5.20 / 31.0	5.21 / 31.0
		B=63	**4.07**×	2.19×	1.12×	0.54×	0.31×	0.24×	5.48 / 63.0	5.47 / 63.0	5.50 / 63.0	5.50 / 63.0	5.47 / 63.0	5.47 / 63.0
	CaDDTree	B=15	2.74×	2.58×	2.23×	1.85×	1.38×	1.22×	4.80 / 15.0	4.78 / 15.0	4.81 / 15.0	4.80 / 15.0	4.78 / 15.0	4.81 / 15.0
		B=31	2.94×	2.63×	2.08×	1.56×	1.12×	0.90×	5.18 / 31.0	5.17 / 31.0	5.16 / 31.0	5.16 / 31.0	5.21 / 31.0	5.17 / 31.0
		B=63	2.95×	2.24×	1.57×	1.05×	0.72×	0.58×	5.46 / 58.9	5.45 / 58.9	5.46 / 58.9	5.45 / 58.9	5.46 / 58.8	5.47 / 58.8
		B=127	2.76×	1.94×	1.30×	0.86×	0.58×	0.46×	5.57 / 81.2	5.54 / 81.4	5.53 / 81.3	5.57 / 81.2	5.58 / 81.2	5.56 / 81.2
	D3Tree	B=15	4.03×	**3.78**×	**3.16**×	**2.63**×	**2.16**×	**1.81**×	4.80 / 15.0	4.82 / 15.0	4.83 / 15.0	4.32 / 8.1	4.16 / 6.5	4.18 / 6.5
G4-E4B	DFlash	N=5	2.82×	2.61×	2.34×	1.97×	1.57×	1.27×	3.49 / 5.0	3.49 / 5.0	3.49 / 5.0	3.50 / 5.0	3.49 / 5.0	3.49 / 5.0
		N=7	3.09×	2.84×	2.48×	2.08×	1.58×	1.23×	3.87 / 7.0	3.85 / 7.0	3.86 / 7.0	3.87 / 7.0	3.87 / 7.0	3.87 / 7.0
		N=15	3.50×	3.10×	2.42×	1.81×	1.26×	0.89×	4.43 / 15.0	4.42 / 15.0	4.43 / 15.0	4.41 / 15.0	4.40 / 15.0	4.41 / 15.0
	DSpark	N=15	3.14×	2.83×	2.37×	1.98×	1.53×	1.14×	4.00 / 5.3	4.02 / 5.3	4.01 / 5.3	4.00 / 5.2	4.04 / 5.3	4.01 / 5.3
	DDTree	B=15	3.72×	2.19×	1.17×	0.56×	0.33×	0.26×	5.13 / 15.0	5.12 / 15.0	5.13 / 15.0	5.13 / 15.0	5.11 / 15.0	5.13 / 15.0
		B=31	**3.87**×	2.23×	1.24×	0.61×	0.37×	0.29×	5.53 / 31.0	5.52 / 31.0	5.53 / 31.0	5.52 / 31.0	5.51 / 31.0	5.52 / 31.0
		B=63	3.78×	1.94×	1.01×	0.56×	0.33×	0.25×	5.82 / 63.0	5.81 / 63.0	5.81 / 63.0	5.82 / 63.0	5.83 / 63.0	5.81 / 63.0
	CaDDTree	B=15	2.61×	2.48×	2.09×	1.70×	1.28×	1.05×	5.12 / 15.0	5.15 / 15.0	5.11 / 15.0	5.11 / 15.0	5.12 / 15.0	5.12 / 15.0
		B=31	2.88×	2.25×	1.77×	1.29×	0.96×	0.76×	5.53 / 30.1	5.51 / 30.2	5.49 / 30.2	5.52 / 30.2	5.49 / 30.2	5.50 / 30.2
		B=63	2.87×	2.07×	1.38×	1.02×	0.71×	0.55×	5.77 / 56.0	5.78 / 56.0	5.77 / 56.0	5.76 / 56.0	5.75 / 56.0	5.74 / 56.2
		B=127	2.42×	1.65×	1.18×	0.82×	0.57×	0.36×	5.87 / 74.3	5.86 / 74.1	5.87 / 74.2	5.86 / 74.2	5.85 / 74.2	5.83 / 74.4
	D3Tree	B=15	3.86×	**3.36**×	**2.71**×	**2.26**×	**1.76**×	**1.41**×	5.12 / 15.0	5.12 / 15.0	5.12 / 15.0	4.60 / 8.0	4.42 / 6.5	4.44 / 6.6

TABLE V

D3Tree at construction budgets $B \in \{15, 31, 63\}$ with the same allocation table, columns as in Table IV: speedup over AR and accepted length τ / verified draft tokens per request and decoding round, over all 17 benchmarks. Best speedup per verifier and column in bold.

Model	Budget	Speedup over AR at concurrency R						Accepted length τ / verified draft tokens at concurrency R					
		1	4	8	16	32	64	1	4	8	16	32	64
Q3.5-4B	B=15	**3.52**×	**3.14**×	**2.66**×	**2.20**×	1.70×	1.37×	5.33 / 15.0	5.35 / 15.0	4.72 / 7.5	4.75 / 7.5	4.52 / 5.9	4.52 / 5.9
	B=31	3.44×	2.80×	2.58×	2.10×	**1.72**×	**1.39**×	5.77 / 31.0	5.77 / 31.0	4.72 / 7.8	4.74 / 7.7	4.52 / 6.0	4.53 / 6.0
	B=63	3.10×	1.81×	2.32×	2.00×	1.67×	1.35×	6.07 / 63.0	6.04 / 63.0	4.74 / 8.2	4.74 / 8.1	4.54 / 6.2	4.54 / 6.2
Q3.5-9B	B=15	**3.73**×	**3.05**×	2.58×	**2.17**×	1.60×	1.28×	5.20 / 15.0	5.20 / 15.0	4.61 / 7.4	4.62 / 7.4	4.41 / 5.8	4.40 / 5.9
	B=31	3.63×	2.73×	**2.61**×	2.12×	**1.63**×	**1.30**×	5.65 / 31.0	5.66 / 31.0	4.62 / 7.6	4.62 / 7.6	4.40 / 6.0	4.40 / 6.0
	B=63	3.20×	1.72×	2.39×	2.05×	1.58×	1.20×	5.97 / 63.0	5.99 / 63.0	4.61 / 7.9	4.64 / 7.9	4.40 / 6.1	4.42 / 6.3
IVL3.5-4B	B=15	3.17×	**2.88**×	**2.46**×	**2.07**×	**1.66**×	1.38×	4.79 / 15.0	4.81 / 15.0	4.80 / 15.0	4.39 / 8.5	4.19 / 6.8	4.21 / 6.9
	B=31	3.24×	2.69×	1.80×	2.02×	**1.66**×	**1.40**×	5.15 / 31.0	5.20 / 31.0	5.17 / 31.0	4.40 / 9.6	4.25 / 7.6	4.24 / 7.8
	B=63	**3.25**×	1.82×	1.27×	1.89×	1.49×	1.28×	5.49 / 63.0	5.47 / 63.0	5.49 / 63.0	4.40 / 11.9	4.22 / 9.6	4.24 / 9.6
IVL3.5-8B	B=15	3.66×	**3.10**×	**2.69**×	**2.25**×	**1.74**×	**1.48**×	5.08 / 15.0	5.06 / 15.0	5.06 / 15.0	4.63 / 8.2	4.45 / 6.8	4.47 / 7.0
	B=31	**3.73**×	2.94×	1.97×	2.16×	1.72×	1.46×	5.47 / 31.0	5.48 / 31.0	5.46 / 31.0	4.62 / 9.2	4.45 / 7.7	4.49 / 8.0
	B=63	3.54×	2.11×	1.53×	2.09×	1.60×	1.40×	5.72 / 63.0	5.80 / 63.0	5.72 / 63.0	4.65 / 11.1	4.46 / 9.3	4.46 / 9.5
G4-E2B	B=15	4.03×	**3.78**×	**3.16**×	**2.63**×	**2.16**×	**1.81**×	4.80 / 15.0	4.82 / 15.0	4.83 / 15.0	4.32 / 8.1	4.16 / 6.5	4.18 / 6.5
	B=31	4.17×	3.65×	2.64×	2.60×	2.12×	1.80×	5.17 / 31.0	5.22 / 31.0	5.19 / 31.0	4.36 / 8.8	4.18 / 7.0	4.20 / 7.1
	B=63	**4.53**×	2.92×	1.84×	2.49×	1.90×	1.68×	5.47 / 63.0	5.47 / 63.0	5.48 / 63.0	4.34 / 10.1	4.20 / 8.1	4.19 / 8.2
G4-E4B	B=15	3.86×	**3.36**×	**2.71**×	2.26×	1.76×	**1.41**×	5.12 / 15.0	5.12 / 15.0	5.12 / 15.0	4.60 / 8.0	4.42 / 6.5	4.44 / 6.6
	B=31	**4.05**×	3.10×	2.26×	**2.29**×	**1.77**×	**1.41**×	5.54 / 31.0	5.51 / 31.0	5.53 / 31.0	4.60 / 8.7	4.42 / 7.1	4.43 / 7.3
	B=63	4.01×	2.41×	1.50×	2.19×	1.59×	1.33×	5.82 / 63.0	5.83 / 63.0	5.82 / 63.0	4.62 / 10.0	4.44 / 8.1	4.44 / 8.4

accepted length.

D3Tree is faster than the best CaDDTree budget at every verifier and load, by 1.23–1.47× at $R = 1$, 1.34–1.48× at $R = 64$, and at least 1.15× at any load. Part of the gap at light load is implementation overhead. Our CaDDTree shares the tree builder, verification kernels, and runtime of D3Tree, but computes its per-request cutoff with unfused tensor operations instead of the fused pruning kernel; at $R = 1$ the median round on Q3.5-4B takes 23.0 ms with CaDDTree $B = 15$, 17.7 ms with DDTree $B = 15$, and 15.4 ms with D3Tree, while all three verify the same trees. Under load, the gap reflects the selection rule: CaDDTree keeps verifying the trees that are optimal for a single request, whereas D3Tree spends a batch-wide budget sized to the current load.

D. Serving over Time

The speedup tables summarize each run by one number per load. To see how the strategies behave within a run, the scheduler records, at every decoding step, the requests in flight and the draft tokens each of them verifies. Fig. 5 first serves 200 requests drawn from all 17 benchmarks at a fixed concurrency on Q3.5-9B and an RTX A6000: all are due at once, 64 are in flight, and a finished request is replaced immediately, as in the sweeps of Table III. D3Tree finishes all 200 after 131.8 s, at 1,301 output tokens/s, ahead of DSpark (134.7 s, 1,270 tokens/s) and AR (139.0 s, 1,223 tokens/s); CaDDTree, DFlash, and DDTree need 192–215 s (788–884 tokens/s), because each of their passes processes about 1,010 tokens against 382 for D3Tree. Its mean latency from sending to the last token is 36.3 s, against 36.8 s for DSpark, 40.0 s for AR, and 53.0–61.3 s for the fixed-width strategies; AR, which advances every request by one token per pass, has the narrowest spread and the lowest 90th percentile (50.6 s, against 57.3 s for D3Tree and 57.5 s for DSpark). The tail of the run shows the allocation at work (Fig. 5(a)): while 64 requests are in flight D3Tree verifies about 5 draft tokens per request, as DSpark does, but as the last requests finish it widens again, to 7 per request below 24 requests in flight and to whole 15-node trees for the last four, whereas DSpark keeps its short prefixes.

A fixed concurrency holds R constant, whereas in serving the load changes within seconds. Fig. 6 therefore replays one arrival trace against every strategy on the same verifier and GPU: 420 requests arrive as a Poisson process whose rate changes over seven phases, from 0.6 to 10 requests per second, and at most 64 requests are in flight, later arrivals waiting at the client. The trace exceeds what a single A6000 serves with this verifier under every strategy: from the burst phase on, 64 requests stay in flight and the rest queue at the client, so the end-to-end latency is set mostly by how fast each strategy drains the queue.

D3Tree adapts the width of every step to the load (Fig. 6(a)). With at most four requests in flight it verifies whole 15-node trees, which this hybrid target can afford; as the concurrency rises it narrows to 7 and then to about 5 draft tokens per request, and it widens again whenever the batch thins out. DFlash, DDTree, and CaDDTree verify 15 draft tokens per request throughout, so with 64 requests in flight each of their target passes processes about 1,010 tokens, against 383 for D3Tree and 365 for DSpark. D3Tree thus keeps the better operating point at each end (Fig. 6(b)). With at most 8 requests in flight it accepts 4.42 tokens per round from 10.9 verified draft tokens, close to the full trees (4.52–4.57 from 15) and 0.78 more than DSpark (3.64). With 16 or more it verifies 5.2 draft tokens per request, about as many as DSpark's 4.8, yet accepts 4.09 against 3.75, because it keeps the most promising nodes of the whole batch rather than a prefix of each chain.

As a result, D3Tree serves the trace with the highest output throughput, 1,161 tokens/s, against 1,145 for DSpark and 1,057 for AR; DFlash, CaDDTree, and DDTree (813, 832, and 733 tokens/s) fall below AR because their 1,010-token passes cost more than their acceptance returns. D3Tree has the lowest mean latency in all seven arrival phases (Fig. 6(c)): 4–13% below DSpark, 6–65% below DFlash, and 25–63% below CaDDTree. Over the trace its mean end-to-end latency

TABLE VI

Speedup over AR and Avg. τ on an RTX 5090 (32 GB) and an A800 (80 GB), with the protocol of Table III. On the A800, D3Tree uses an allocation table retuned for that GPU at $R \geq 16$.

GPU	Model	Strategy	Budget	Text benchmarks							Multimodal benchmarks						
				R=1	R=4	R=8	R=16	R=32	R=64	Avg. τ	R=1	R=4	R=8	R=16	R=32	R=64	Avg. τ
RTX 5090	Q3.5-4B	DFlash	N=15	3.02×	3.18×	2.90×	2.06×	1.35×	0.98×	5.35	2.21×	2.30×	2.10×	1.52×	1.01×	0.79×	4.05
		DSpark		2.79×	2.94×	2.92×	2.32×	1.69×	1.33×	4.94	2.04×	2.11×	2.16×	1.76×	1.32×	**1.11**×	3.66
		DDTree		3.12×	3.15×	2.81×	1.79×	1.10×	0.83×	6.15	2.36×	2.40×	2.10×	1.41×	0.89×	0.68×	4.75
		CaDDTree	B=15	2.31×	2.49×	2.33×	1.74×	1.22×	0.95×	6.15	1.78×	1.86×	1.74×	1.34×	0.95×	0.76×	4.75
		D3Tree		**3.62**×	**3.61**×	**3.18**×	**2.43**×	**1.72**×	**1.35**×	5.56	**2.66**×	**2.61**×	**2.35**×	**1.83**×	**1.35**×	1.08×	4.38
	IVL3.5-4B	DFlash	N=15	3.05×	3.40×	3.10×	2.12×	1.41×	1.01×	4.68	2.03×	2.13×	1.90×	1.44×	1.17×	0.98×	3.71
		DSpark		2.73×	3.17×	3.14×	2.51×	1.75×	1.28×	4.34	1.76×	1.93×	1.86×	1.60×	1.35×	1.18×	3.29
		DDTree		3.03×	2.99×	2.28×	1.28×	0.70×	0.57×	5.44	1.92×	1.78×	1.49×	1.04×	0.73×	0.65×	4.35
		CaDDTree	B=15	2.18×	2.53×	2.43×	1.86×	1.30×	0.99×	5.45	1.52×	1.66×	1.55×	1.25×	1.06×	1.00×	4.34
		D3Tree		**3.36**×	**3.79**×	**3.46**×	**2.72**×	**1.93**×	**1.42**×	5.08	**2.10**×	**2.26**×	**2.04**×	**1.69**×	**1.44**×	**1.28**×	4.17
	G4-E2B	DFlash	N=15	**5.86**×	**5.50**×	**4.53**×	3.30×	2.31×	1.58×	5.35	**3.09**×	2.65×	2.38×	1.71×	1.35×	1.10×	3.26
		DSpark		5.03×	4.78×	4.14×	3.36×	2.56×	1.65×	4.92	2.58×	2.34×	2.28×	1.83×	1.66×	1.37×	2.90
		DDTree		4.85×	3.51×	1.91×	0.87×	0.65×	0.42×	6.12	2.72×	1.79×	1.18×	0.57×	0.30×	0.25×	3.93
		CaDDTree	B=15	3.63×	3.48×	3.04×	2.54×	1.94×	1.41×	6.09	2.23×	2.01×	1.94×	1.48×	1.29×	1.11×	3.91
		D3Tree		5.34×	4.89×	4.47×	**3.59**×	**3.02**×	**2.08**×	5.69	2.95×	**2.76**×	**2.61**×	**2.05**×	**1.88**×	**1.62**×	3.72
A800	Q3.5-4B	DFlash	N=15	3.45×	2.97×	2.55×	1.86×	1.32×	0.95×	5.36	2.52×	2.16×	1.78×	1.34×	1.00×	0.76×	4.05
		DSpark		3.22×	2.89×	2.65×	**2.13**×	**1.67**×	**1.28**×	4.96	2.28×	2.11×	1.89×	**1.60**×	**1.28**×	**1.07**×	3.65
		DDTree		3.50×	2.94×	2.23×	1.43×	0.94×	0.69×	6.17	2.59×	2.13×	1.58×	1.07×	0.73×	0.55×	4.75
		CaDDTree	B=15	2.64×	2.25×	1.97×	1.48×	1.12×	0.83×	6.15	1.96×	1.68×	1.40×	1.12×	0.87×	0.70×	4.73
		D3Tree		**4.07**×	**3.25**×	**2.67**×	2.04×	1.43×	1.18×	5.55	**2.92**×	**2.35**×	**1.92**×	1.55×	1.18×	1.00×	4.35
	Q3.5-9B	DFlash	N=15	3.46×	3.16×	2.73×	1.91×	1.26×	0.91×	5.25	2.54×	2.24×	1.91×	1.38×	0.97×	0.70×	3.93
		DSpark		3.17×	2.97×	2.75×	2.21×	**1.66**×	**1.24**×	4.72	2.30×	2.16×	1.97×	**1.67**×	**1.29**×	**1.03**×	3.52
		DDTree		3.71×	3.17×	2.45×	1.64×	1.04×	0.75×	6.06	2.75×	2.28×	1.79×	1.23×	0.80×	0.58×	4.63
		CaDDTree	B=15	2.87×	2.56×	2.19×	1.66×	1.16×	0.86×	6.04	2.10×	1.84×	1.60×	1.24×	0.90×	0.69×	4.61
		D3Tree		**3.98**×	**3.35**×	**2.81**×	**2.23**×	1.56×	**1.24**×	5.43	**2.87**×	**2.39**×	**2.08**×	**1.67**×	1.24×	0.99×	4.24
	IVL3.5-4B	DFlash	N=15	3.57×	2.81×	2.36×	1.65×	1.38×	1.03×	4.68	2.32×	1.83×	1.51×	1.22×	1.11×	0.96×	3.70
		DSpark		3.29×	2.83×	2.48×	1.93×	1.58×	1.25×	4.32	2.10×	1.77×	1.55×	1.37×	**1.22**×	1.12×	3.29
		DDTree		3.47×	2.50×	1.70×	0.92×	0.60×	0.46×	5.46	2.25×	1.46×	1.06×	0.73×	0.57×	0.48×	4.36
		CaDDTree	B=15	2.55×	2.14×	1.90×	1.43×	1.25×	0.96×	5.44	1.79×	1.48×	1.27×	1.03×	0.98×	0.87×	4.34
		D3Tree		**3.89**×	**3.19**×	**2.69**×	**2.04**×	**1.59**×	**1.28**×	5.15	**2.52**×	**1.95**×	**1.64**×	**1.39**×	1.17×	**1.14**×	4.19
	IVL3.5-8B	DFlash	N=15	4.11×	3.46×	2.84×	1.97×	1.46×	1.07×	5.38	2.40×	1.99×	1.66×	1.34×	1.08×	0.90×	3.69
		DSpark		3.87×	3.37×	2.94×	2.19×	1.72×	1.31×	5.12	2.19×	1.93×	1.72×	1.50×	**1.29**×	**1.14**×	3.37
		DDTree		4.07×	3.12×	2.10×	1.13×	0.70×	0.54×	6.11	2.45×	1.75×	1.29×	0.89×	0.64×	0.52×	4.36
		CaDDTree	B=15	3.18×	2.89×	2.47×	1.78×	1.37×	1.00×	6.09	1.98×	1.75×	1.53×	1.24×	1.07×	0.91×	4.37
		D3Tree		**4.43**×	**3.82**×	**3.16**×	**2.44**×	**1.80**×	**1.47**×	5.76	**2.63**×	**2.18**×	**1.79**×	**1.53**×	**1.29**×	1.13×	4.20
	G4-E2B	DFlash	N=15	5.77×	5.17×	4.22×	3.09×	2.13×	1.41×	5.34	3.14×	2.53×	2.19×	1.74×	1.40×	0.96×	3.24
		DSpark		5.34×	4.90×	3.83×	3.09×	2.28×	1.56×	4.93	2.77×	2.33×	1.99×	1.66×	1.52×	1.25×	2.89
		DDTree		5.56×	3.64×	1.82×	0.84×	0.50×	0.40×	6.11	3.04×	1.77×	1.04×	0.51×	0.28×	0.22×	3.90
		CaDDTree	B=15	3.78×	3.66×	3.19×	2.56×	1.89×	1.39×	6.09	2.28×	2.00×	1.81×	1.57×	1.35×	1.09×	3.91
		D3Tree		**6.03**×	**5.61**×	**4.62**×	**3.66**×	**2.73**×	**2.06**×	5.75	**3.28**×	**2.76**×	**2.43**×	**2.14**×	**1.80**×	**1.47**×	3.75
	G4-E4B	DFlash	N=15	5.09×	4.56×	3.69×	2.60×	1.71×	0.99×	5.82	2.48×	2.09×	1.78×	1.36×	1.16×	0.85×	3.44
		DSpark		4.73×	4.32×	3.37×	2.62×	1.84×	1.11×	5.47	2.17×	1.91×	1.66×	1.41×	1.29×	1.03×	3.01
		DDTree		5.11×	3.25×	1.69×	0.79×	0.51×	0.33×	6.55	2.62×	1.55×	0.88×	0.42×	0.24×	0.20×	4.11
		CaDDTree	B=15	3.71×	3.60×	3.15×	2.41×	1.73×	1.27×	6.54	2.01×	1.79×	1.58×	1.28×	1.14×	0.87×	4.10
		D3Tree		**5.56**×	**5.11**×	**4.21**×	**3.44**×	**2.40**×	**1.73**×	6.18	**2.79**×	**2.39**×	**2.02**×	**1.72**×	**1.45**×	**1.12**×	3.95

is 55.4 s and its 90th percentile 91.1 s, against 58.8 s and 95.9 s for DSpark, 73.8 s for AR, 118.7 s for CaDDTree, 120.7 s for DFlash, and 147.2 s for DDTree. At most 167 requests are in the system with D3Tree, against 173 with DSpark and 233–252 with the fixed-width strategies, and its last request completes 59 s after the last arrival, against 64 s for DSpark and 179–238 s for DFlash, CaDDTree, and DDTree.

E. Construction Budget

Table V runs D3Tree with $B = 31$ and 63 under the allocation table set for $B = 15$. Up to R_{full} D3Tree verifies whole trees, so a larger B raises both the accepted length and the verification width: at $R = 1$, τ grows by 0.36–0.45 tokens

at $B = 31$ and by 0.64–0.77 at $B = 63$. On IVL3.5 and G4 a single request still gains from the larger tree, by up to 12% with $B = 31$ or 63; on the hybrid Qwen3.5 targets $B = 15$ remains fastest. Already at $R = 4$, verifying whole larger trees costs 3–11% of the speedup with $B = 31$ and 23–44% with $B = 63$. Above R_{full} the batch-wide budget N_{tot} caps the verified rows, so the construction budget matters little: from $R = 16$ on, $B = 31$ verifies about one more draft token per tree and stays within 5% of $B = 15$, and $B = 63$ within 12%. A larger construction budget thus helps only a single request on some verifiers and costs speedup at moderate load; we keep $B = 15$.

F. Results on Other GPUs

Table VI repeats the comparison on an RTX 5090, for Q3.5-4B, IVL3.5-4B, and G4-E2B, and on an A800, for all six verifiers. On the RTX 5090 the allocation table is unchanged. On the A800, whose tiles differ from those of the RTX 6000 Ada, we retuned the per-request allocation for $R \geq 16$ on six of the 17 benchmarks with 48 prompts each: dense targets keep $\bar{n} = 7$ at every $R \geq 9$, and hybrid targets drop to $\bar{n} = 4$ from $R = 24$; $R \leq 8$ is unchanged. Against the table of Section III-E, this changes the speedup at $R \geq 16$ by -5% to $+7\%$.

On the RTX 5090, D3Tree is the fastest strategy in 31 of the 36 settings and 1.28–1.57× as fast as CaDDTree in every setting. On the smallest verifier, G4-E2B, DFlash's full linear draft is ahead up to $R = 8$ on the text benchmarks (by up to 0.61×, at $R = 4$) and at $R = 1$ on the multimodal ones (3.09× vs. 2.95×), and D3Tree leads from $R = 16$ on; the remaining exception is DSpark on the multimodal benchmarks of Q3.5-4B at $R = 64$ (1.11× vs. 1.08×). At $R = 64$, D3Tree reaches 1.21–1.87× over AR, whereas DDTree falls to 0.34–0.75×.

On the A800, D3Tree is the fastest strategy in 61 of the 72 settings, reaching 3.14–4.48× over AR for a single request and 1.08–1.79× at $R = 64$, where DDTree falls to 0.27–0.66×. It is 1.17–1.59× as fast as CaDDTree in every setting. All eleven exceptions are DSpark: on the hybrid Qwen3.5 targets from $R = 16$ on (nine settings, by up to 0.24×; Q3.5-4B, text, $R = 32$: 1.67× vs. 1.43×), and on the multimodal benchmarks of IVL3.5-4B at $R = 32$ (0.05×) and IVL3.5-8B at $R = 64$ (0.01×). On Qwen3.5 the gap does not close with the allocation: in the retuning, D3Tree stayed behind DSpark on Q3.5-4B even when it verified as few draft tokens per request, so it comes from the per-step cost of tree verification on this GPU, the tree attention and the replay of the GDN states, rather than from the number of verified tokens.

V. Conclusion

We presented D3Tree, which decides how much of a block-diffusion draft tree to verify. A tree-size head with $2H+2$ parameters scores every candidate, parent-chain products yield an ancestor-closed subtree, and a batch-wide node budget aligned with the target's matrix-multiplication tiles sets how many nodes each decoding step verifies. Under light load, D3Tree verifies whole trees; as concurrency grows, it spends one budget across the batch instead of a tree size fixed per request in advance. On six vision-language verifiers and 17 text and multimodal benchmarks, D3Tree is the fastest of the compared verification strategies in 70 of 72 load settings, reaching 3.2–4.0× over autoregressive decoding for a single request and 1.3–1.8× at 64 concurrent requests, where fixed-budget trees fall below autoregressive speed. In serving, it completes a fixed-concurrency workload first and attains the lowest mean latency in every phase of a time-varying arrival trace.

Two limitations remain. On the A800, the per-step cost of tree verification on hybrid targets, namely tree attention and the replay of recurrent states, lets dynamically truncated chains overtake D3Tree under heavy load; closing this gap calls for cheaper tree kernels rather than a different allocation. In addition, our verification is greedy. Extending tree pruning to stochastic sampling, and evaluating the optional joint training of the drafter at scale, are left to future work.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are Few-Shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix *et al.*, "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.

[4] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng *et al.*, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.

[5] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya *et al.*, "Efficiently scaling transformer inference," in *Proc. Mach. Learn. Syst. (MLSys)*, 2023.

[6] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, vol. 202, 2023, pp. 19 274–19 286.

[7] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv preprint arXiv:2302.01318, 2023.

[8] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block diffusion for flash speculative decoding," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, vol. 306, 2026, pp. 15 343–15 355.

[9] L. Ringel and Y. Romano, "Accelerating speculative decoding with block diffusion draft trees," *arXiv preprint arXiv:2604.12989*, 2026.

[10] S. Zhang, H. Qiu, H. He, and Y. Dai, "Cost-aware diffusion draft trees for speculative decoding," *arXiv preprint arXiv:2606.01813*, 2026.

[11] X. Cheng, X. Yu, C. Shao, J. Li, Y. Xiong, Y. Qian *et al.*, "DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation," *arXiv preprint arXiv:2607.05147*, 2026.

[12] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018.

[13] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui, "Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation," in *Findings Assoc. Comput. Linguistics: EMNLP*, 2023.

[14] Y. Zhou, K. Lyu, A. S. Rawat, A. K. Menon, A. Rostamizadeh, S. Kumar *et al.*, "DistillSpec: Improving speculative decoding via knowledge distillation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[15] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica *et al.*, "Online speculative decoding," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[16] S. Kim, K. Mangalam, S. Moon, J. Malik, M. W. Mahoney, A. Gholami *et al.*, "Speculative decoding with big little decoder," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[17] B. Spector and C. Re, "Accelerating LLM inference with staged speculative decoding," arXiv preprint arXiv:2308.04623, 2023.

[18] Z. Chen, X. Yang, J. Lin, C. Sun, K. C.-C. Chang, and J. Huang, "Cascade speculative drafting for even faster LLM inference," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[19] W. Zhao, Y. Huang, X. Han, W. Xu, C. Xiao, X. Zhang *et al.*, "Ouroboros: Generating longer drafts phrase by phrase for faster speculative decoding," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2024.

[20] Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He, "REST: Retrieval-based speculative decoding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL)*, 2024.

[21] N. Yang, T. Ge, L. Wang, B. Jiao, D. Jiang, L. Yang *et al.*, "Inference with reference: Lossless acceleration of large language models," arXiv preprint arXiv:2304.04487, 2023.

[22] A. Santilli, S. Severino, E. Postolache, V. Maiorca, M. Mancusi, R. Marin *et al.*, "Accelerating transformer inference for translation via parallel decoding," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2023.

[23] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of LLM inference using lookahead decoding," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[24] S. Kou, L. Hu, Z. He, Z. Deng, and H. Zhang, "CLLMs: Consistency large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[25] Z. Sun, A. T. Suresh, J. H. Ro, A. Beirami, H. Jain, and F. Yu, "SpecTr: Fast speculative decoding via optimal transport," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[26] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge *et al.*, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings Assoc. Comput. Linguistics: ACL*, 2024.

[27] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding speculative decoding," in *Proc. Conf. Nations Americas Chapter Assoc. Comput. Linguistics (NAACL)*, 2025.

[28] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen *et al.*, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[29] Z. Ankner, R. Parthasarathy, A. Nrusimha, C. Rinard, J. Ragan-Kelley, and W. Brandon, "Hydra: Sequentially-dependent draft heads for Medusa decoding," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[30] Y. Cheng, A. Zhang, X. Zhang, C. Wang, and Y. Wang, "Recurrent drafter for fast speculative decoding in large language models," arXiv preprint arXiv:2403.09919, 2024.

[31] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[32] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

[33] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, "Better & faster large language models via multi-token prediction," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[34] DeepSeek-AI, "DeepSeek-V3 technical report," arXiv preprint arXiv:2412.19437, 2024.

[35] N. Bhendawade, I. Belousova, Q. Fu, H. Mason, M. Rastegari, and M. Najibi, "Speculative streaming: Fast LLM inference without auxiliary models," in *Proc. 4th NeurIPS Efficient Natural Lang. Speech Process. Workshop (ENLSP)*, vol. 262, 2024, pp. 395–413.

[36] J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen *et al.*, "Draft & verify: Lossless large language model acceleration via self-speculative decoding," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[37] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai *et al.*, "LayerSkip: Enabling early exit inference and self-speculative decoding," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[38] F. Liu, Y. Tang, Z. Liu, Y. Ni, K. Han, and Y. Wang, "Kangaroo: Lossless self-speculative decoding for accelerating LLMs via double early exiting," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[39] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, "Structured denoising diffusion models in discrete state-spaces," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.

[40] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu *et al.*, "Simple and effective masked diffusion language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[41] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu *et al.*, "Large language diffusion models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

[42] M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han *et al.*, "Block diffusion: Interpolating between autoregressive and diffusion language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[43] J. K. Christopher, B. R. Bartoldson, T. Ben-Nun, M. Cardei, B. Kailkhura, and F. Fioretto, "Speculative diffusion decoding: Accelerating language generation through diffusion," in *Proc. Conf. Nations Americas Chapter Assoc. Comput. Linguistics (NAACL)*, 2025.

[44] T. Wu, Y. Yao, Z. Qi, H. Zheng, Z. Wang, H. Ma *et al.*, "D-PACE: Dynamic position-aware cross-entropy for parallel speculative drafting," *arXiv preprint arXiv:2605.18810*, 2026.

[45] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang *et al.*, "SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification," in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2024.

[46] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia *et al.*, "Sequoia: Scalable and robust speculative decoding," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 129 531–129 563.

[47] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-2: Faster inference of language models with dynamic draft trees," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2024.

[48] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan *et al.*, "OPT-Tree: Speculative decoding with adaptive draft tree structure," *Trans. Assoc. Comput. Linguistics*, 2025.

[49] R. Svirschevski, A. May, Z. Chen, B. Chen, Z. Jia, and M. Ryabinin, "SpecExec: Massively parallel speculative decoding for interactive LLM inference on consumer devices," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[50] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat *et al.*, "Dynamic speculation lookahead accelerates speculative decoding of large language models," in *Proc. 4th NeurIPS Efficient Natural Lang. Speech Process. Workshop (ENLSP)*, vol. 262, 2024, pp. 456–467.

[51] K. Huang, X. Guo, and M. Wang, "SpecDec++: Boosting speculative decoding via adaptive candidate lengths," in *Proc. Conf. Lang. Model. (COLM)*, 2025.

[52] T. Liu, Y. Li, Q. Lv, K. Liu, J. Zhu, W. Hu *et al.*, "PEARL: Parallel speculative decoding with adaptive draft length," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[53] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-based generative models," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2022.

[54] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symp. Oper. Syst. Princ. (SOSP)*, 2023.

[55] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani *et al.*, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2024.

[56] Q. Su, C. Giannoula, and G. Pekhimenko, "The synergy of speculative decoding and batching in serving large language models," arXiv preprint arXiv:2310.18813, 2023.

[57] X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang *et al.*, "TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput," arXiv preprint arXiv:2406.14066, 2024.

[58] R. Sadhukhan, J. Chen, Z. Chen, V. Tiwari, R. Lai, J. Shi *et al.*, "MagicDec: Breaking the Latency-Throughput tradeoff for long context generation with speculative decoding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[59] H. Sun, Z. Chen, X. Yang, Y. Tian, and B. Chen, "TriForce: Lossless acceleration of long sequence generation with hierarchical speculative decoding," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[60] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving Mamba2 with delta rule," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[61] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, 2024.

[62] J. Dong, B. Feng, D. Guessous, Y. Liang, and H. He, "FlexAttention: A programming model for generating fused attention variants," in *Proc. Mach. Learn. Syst. (MLSys)*, 2025.

[63] A. Gu and T. Dao, "Mamba: Linear-Time sequence modeling with selective state spaces," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[64] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-Awareness," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.

[65] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[66] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[67] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang *et al.*, "Measuring mathematical problem solving with the MATH dataset," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, vol. 34, 2021.

[68] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee *et al.*, "Let's verify step by step," in *International Conference on Learning Representations*, 2024.

[69] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[70] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[71] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang *et al.*, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," in *International Conference on Learning Representations*, 2025.

[72] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang *et al.*, "Judging LLM-as-a-Judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, vol. 36, 2023.

[73] L. Ouyang, Y. Qu, H. Zhou, J. Zhu, R. Zhang, Q. Lin *et al.*, "OmniDocBench: Benchmarking diverse pdf document parsing with comprehensive annotations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 24 838–24 848.

[74] L. Fu, Z. Kuang, J. Song, M. Huang, B. Yang, Y. Li *et al.*, "OCRBench v2: An improved benchmark for evaluating large multimodal models on visual text localization and reasoning," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, vol. 38, 2025.

[75] M. Mathew, D. Karatzas, and C. V. Jawahar, "DocVQA: A dataset for VQA on document images," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021, pp. 2200–2209.

[76] J. Poznanski, A. Rangapur, J. Borchardt, J. Dunkelberger, R. Huff, D. Lin *et al.*, "olmOCR: Unlocking trillions of tokens in PDFs with vision language models," *arXiv preprint arXiv:2502.18443*, 2025.

[77] Z. Wang, M. Xia, L. He, H. Chen, Y. Liu, R. Zhu *et al.*, "CharXiv: Charting gaps in realistic chart understanding in multimodal LLMs," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, vol. 37, 2024.

[78] A. Singh, V. Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra *et al.*, "Towards VQA models that can read," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 8317–8326.

[79] X. Yue, T. Zheng, Y. Ni, Y. Wang, K. Zhang, S. Tong *et al.*, "MMMU-Pro: A more robust multi-discipline multimodal understanding benchmark," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 15 134–15 186.

[80] D. A. Hudson and C. D. Manning, "GQA: A new dataset for real-world visual reasoning and compositional question answering," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 6700–6709.

[81] X. Chen, H. Fang, T.-Y. Lin, R. Vedantam, S. Gupta, P. Dollar *et al.*, "Microsoft COCO captions: Data collection and evaluation server," *arXiv preprint arXiv:1504.00325*, 2015.

[82] P. Agrawal, S. Antoniak, E. B. Hanna, B. Bout, D. Chaplot, J. Chudnovsky *et al.*, "Pixtral 12B," *arXiv preprint arXiv:2410.07073*, 2024.